

- ❑ 基于推理的方法和神经网络
 - ❑ 基于计数的方法的问题
 - ❑ 基于推理的方法的概要
 - ❑ 神经网络中单词的处理方法
- ❑ 简单的word2vec
 - ❑ CBOW模型的推理
 - ❑ CBOW模型的学习
 - ❑ Word2vec的权重和分布式表示
- ❑ 学习数据的准备
 - ❑ 上下文和目标词
 - ❑ 转化为one-hot表示

Word2vec高速化

- 改进一
 - Embedding层
- 改进二
 - 中间层之后的计算问题
 - 从多分类到二分类
 - Sigmoid函数和交叉熵误差
 - 负采样
 - 负采样的采样方法
- word2vec的应用

基于推理的方法和神经网络方法

用向量表示单词(分布式假设)

①基于计数的方法

②基于推理的方法

基于计数的方法的问题

- ❑ 基于计数的方法根据一个单词周围的单词的出现频数来表示该单词
 - ① 生成所有单词的共现矩阵
 - ② 对这个矩阵进行PPMI和SVD，以获得密集向量（单词的分布式表示）
- ❑ 问题
 - ▣ 处理大规模语料库

基于计数的方法的问题

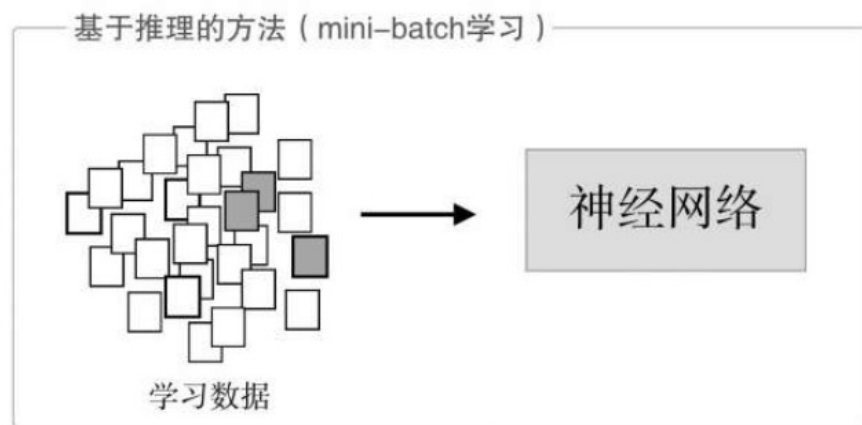
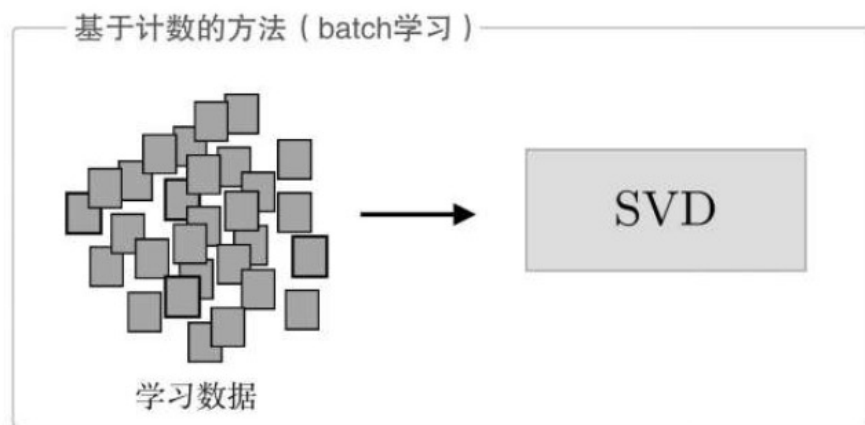
□ 问题

□ 语料库处理的单词数量非常大

- 英文的词汇量超过100万个
- 基于计数的方法就需要生成一个100万×100万的庞大矩阵
- 如此庞大的矩阵执行SVD显然是不现实的
- 对于一个 $n \times n$ 的矩阵，SVD的复杂度是 $O(n^3)$ ，这表示计算量与 n 的立方成比例增长

基于计数的方法的问题

基于统计计数	基于推理
使用整个语料库的统计数据(共现矩阵和PPMI等)，通过一次处理(SVD等)获得单词的分布式表示	使用神经网络，通常在mini-batch数据上进行学习
一次性处理全部学习数据	部分学习数据逐步学习
词汇量很大的语料库中，使SVD等的计算量太大导致计算机难以处理	神经网络可以在部分数据上学习，可并行



基于推理的方法的概要

- ❑ 基于推理的方法的主要操作：“推理”
- ❑ 推理：给出周围的单词(上下文)时，预测 “?” 处会出现什么单词



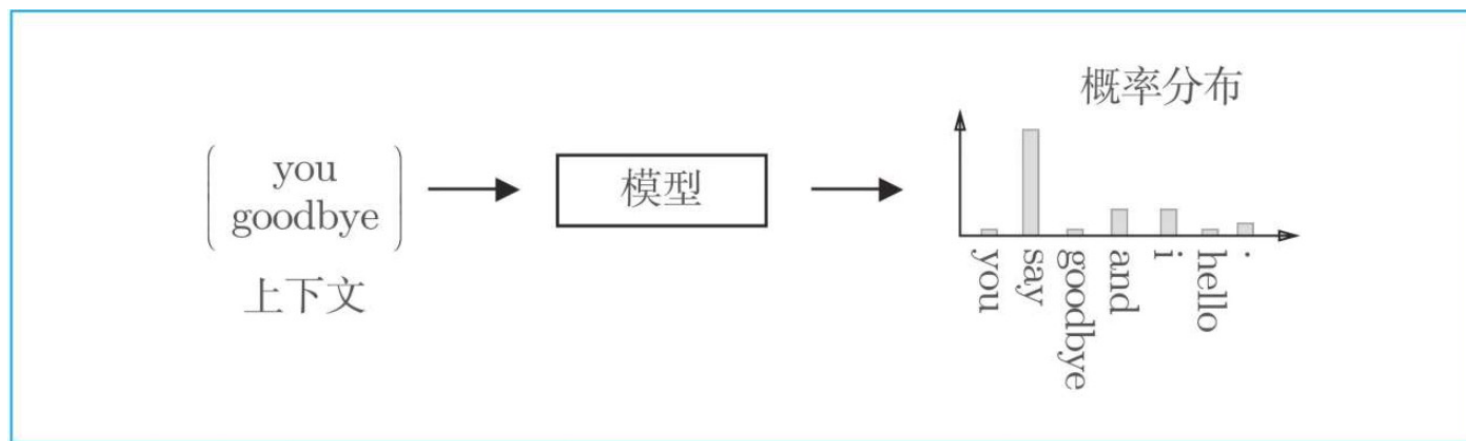
you ? goodbye and i say hello.

- ❑ 基于推理的方法的主要任务：解开推理问题并学习规律
- ❑ 通过反复求解这些推理问题，可以学习到单词的出现模式
- ❑ 从“模型视角”出发，这个推理问题如下

基于推理的方法的概要

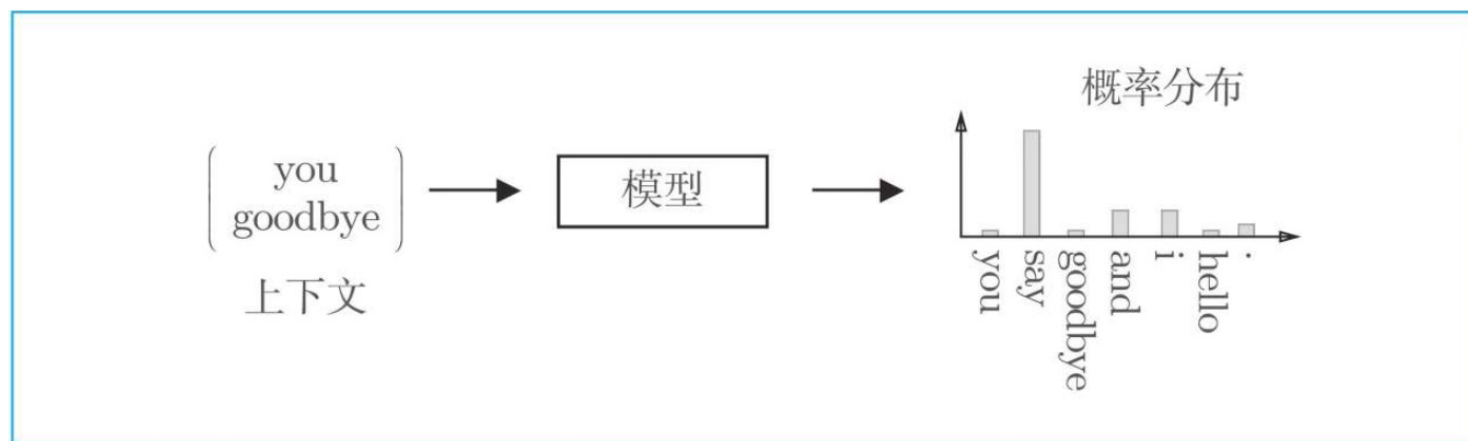
基于推理的方法的全貌

- 输入上下文，模型输出各个单词的出现概率



- 基于推理的方法引入了某种模型，将神经网络用于此模型
- 这个模型接收上下文信息作为输入，输出各个单词的出现概率
- 使用语料库来学习模型，使之能做出正确的预测。
- 模型学习的产物：单词的分布式表示

基于推理的方法的概要



- ❑ 基于推理的方法和基于计数的方法一样，也基于分布式假设“单词含义由其周围的单词构成”
- ❑ 基于推理的方法将这一假设归结为了上面的预测问题。
- ❑ 不管是哪种方法，如何对基于分布式假设的“单词共现”建模都是最重要的研究主题

神经网络中单词的处理方法

- 从现在开始，我们将使用神经网络来处理单词
- 但是，神经网络无法直接处理you或say这样的单词，要用神经网络处理单词，需要先将单词转化为固定长度的向量
- 一种方式是将单词转换为one-hot表示（one-hot向量）。在one-hot表示中，只有一个元素是1，其他元素都是0。

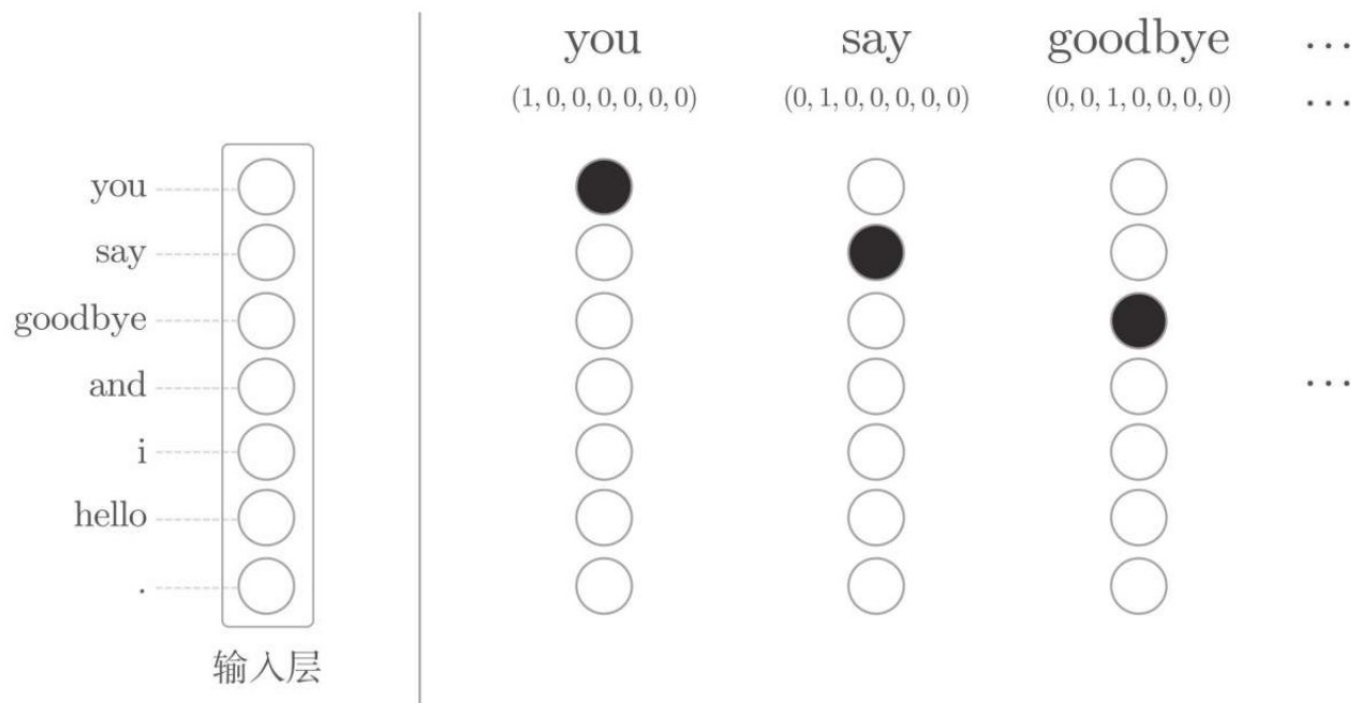
神经网络中单词的处理方法

- one-hot表示的例子：“You say goodbye and I say hello.”
- 在这个语料库中，一共有7个单词（“you”“say”“goodbye”“and”“i”“hello”“.”）。此时，各个单词可以转化为下图所示的one-hot表示。

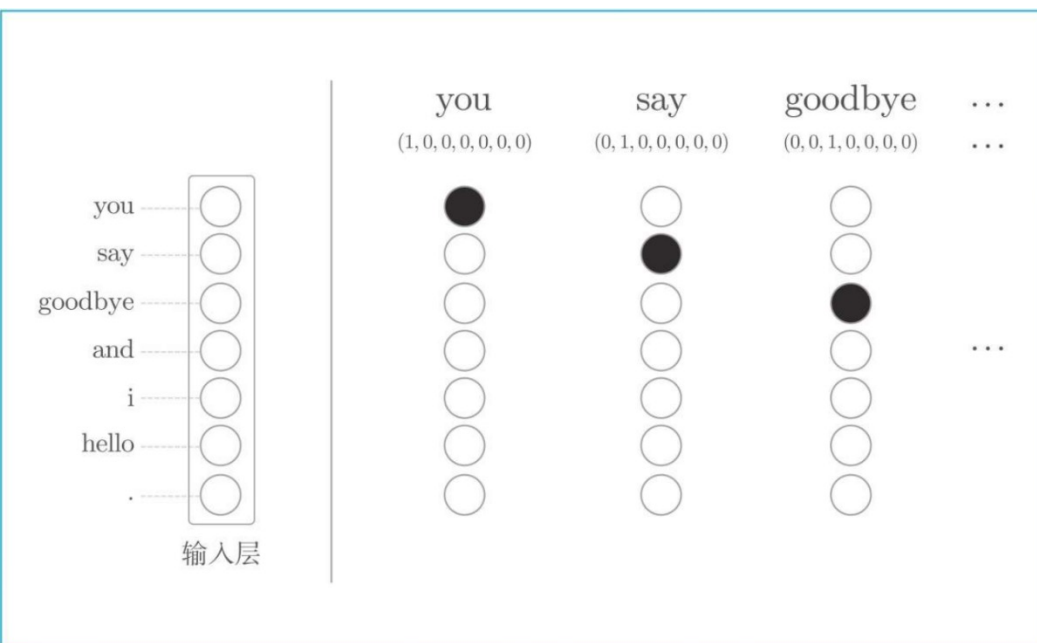
单词	单词ID	one-hot表示
$\begin{pmatrix} \text{you} \\ \text{goodbye} \end{pmatrix}$	$\begin{pmatrix} 0 \\ 2 \end{pmatrix}$	$\begin{pmatrix} (1, 0, 0, 0, 0, 0, 0) \\ (0, 0, 1, 0, 0, 0, 0) \end{pmatrix}$

神经网络中单词的处理方法

- 单词可以表示为文本、单词ID和one-hot表示
- 要将单词转化为one-hot表示，就需要准备元素个数与词汇个数相等的向量，并将单词ID对应的元素设为1，其他元素设为0
- 将单词转化为固定长度的向量 神经网络的输入层的神经元个数就可以固定



神经网络中单词的处理方法

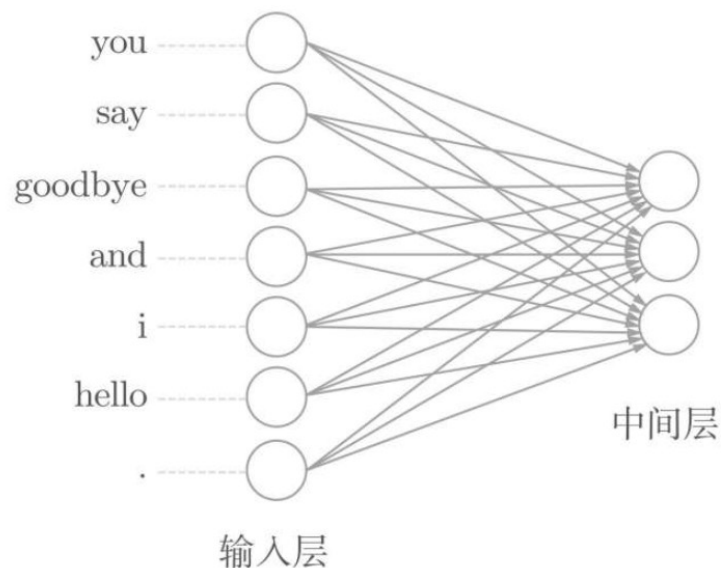


- 各个**神经元**对应于各个单词。神经元为1的地方用黑色绘制，为0的地方用白色绘制
- **输入层**由7个神经元表示，分别对应于7个单词（第1个神经元对应于you，第2个神经元对应于say）。

- 将单词表示为向量，这些向量就可以由构成神经网络的各种“层”来处理。比如，对于one-hot表示的某个单词，使用全连接层对其进行变换的情况如下图所示

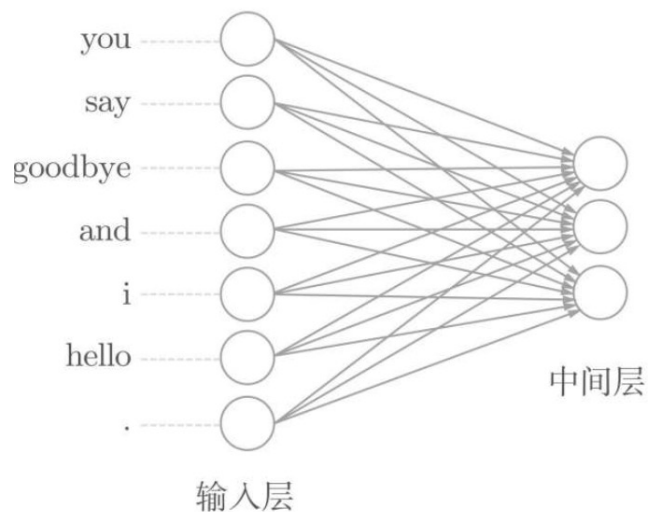
神经网络中单词的处理方法

- 基于神经网络的全连接层的变换：输入层的各个神经元分别对应于7个单词（中间层的神经元暂为3个）
- 全连接层通过箭头连接所有节点。箭头拥有权重（参数），它们和输入层神经元的加权和成为中间层的神经元。使用的全连接层将省略偏置

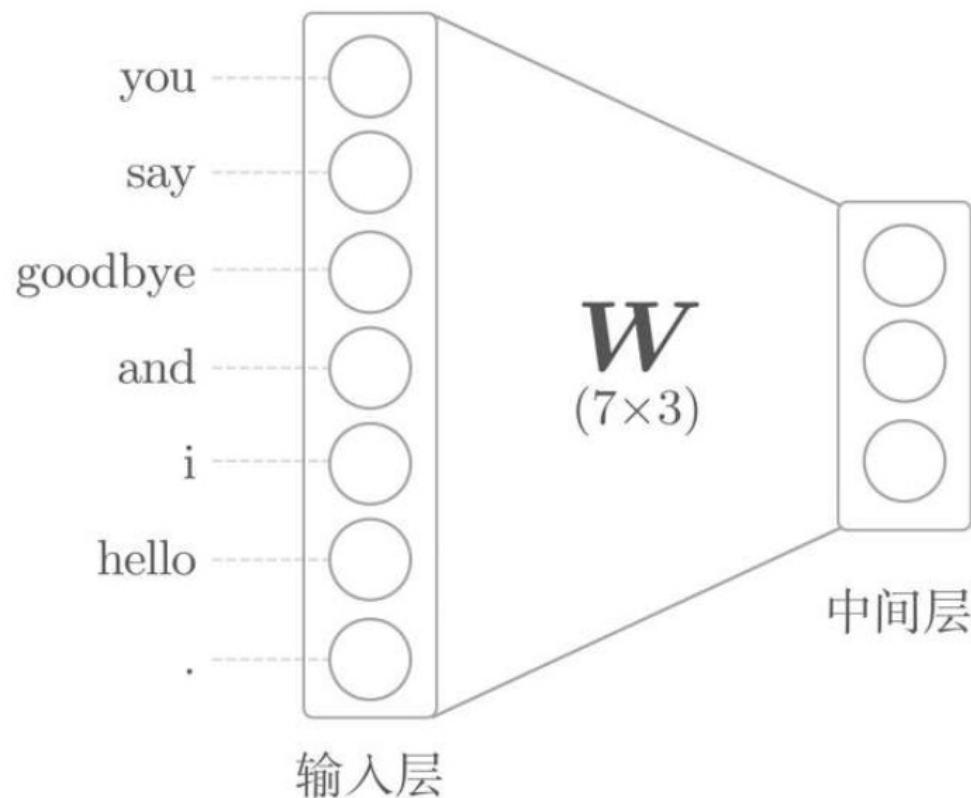


神经网络中单词的处理方法

- 神经元之间的连接是用箭头表示的

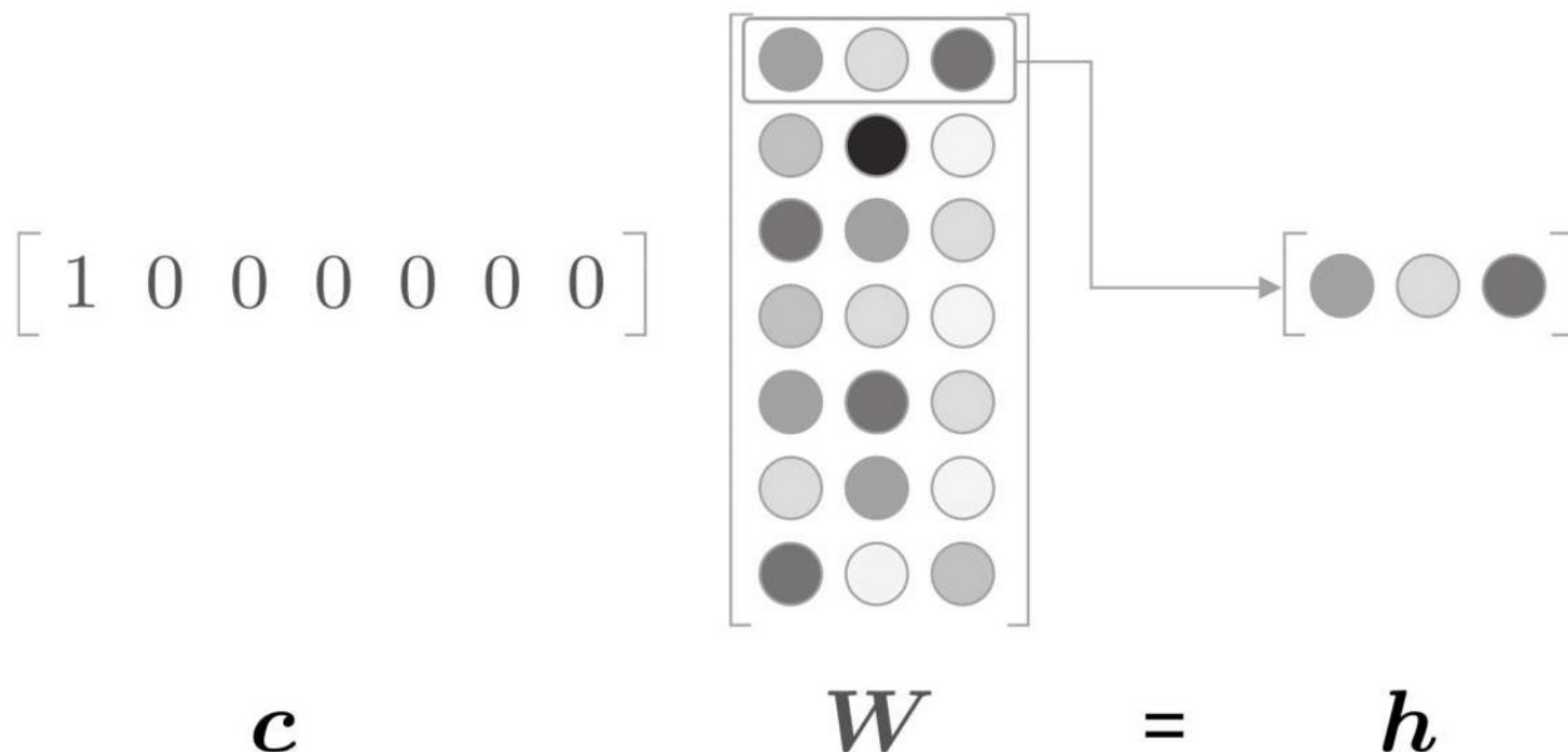


- 为了明确地显示权重
- 将全连接层的权重表示为一个 7×3 形状的 W 矩阵



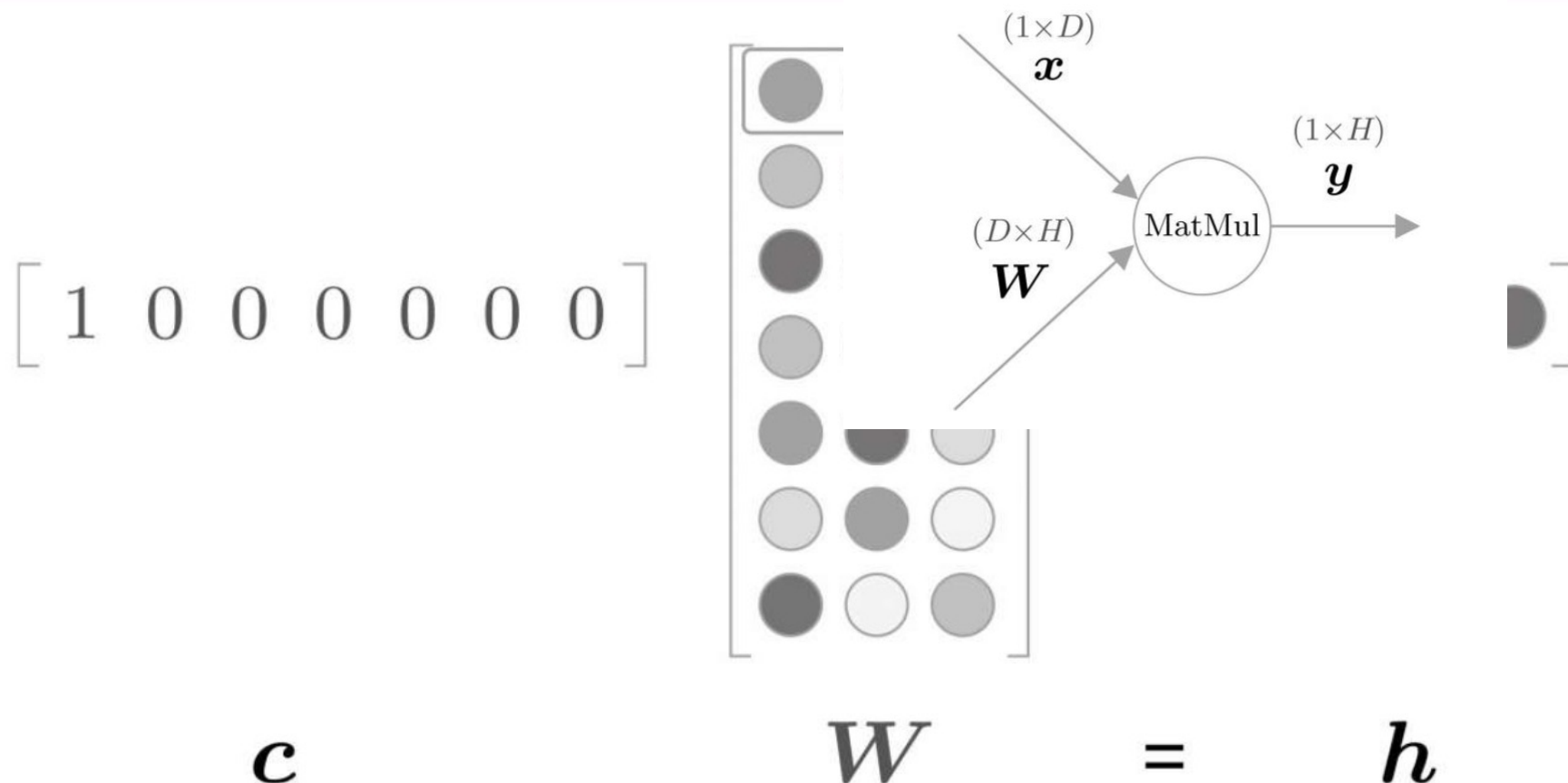
神经网络中单词的处理方法

- 在上下文 c 和权重 W 的矩阵乘积中，对应位置的行向量被提取(权重的各个元素的大小用灰度表示)



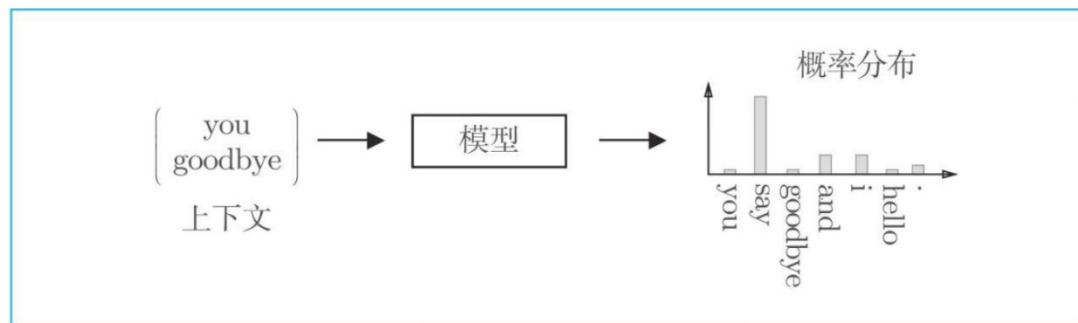
神经网络中单词的处理方法

- 在上下文 c 和权重 W 的矩阵乘积中，对应位置的行向量被提取(权重的各个元素的大小用灰度表示)



简单的word2vec

- ❑ 基于推理的方法，并讨论了神经网络中单词的处理方法，准备工作完成，现在是时候实现word2vec了
- ❑ 要做的事情是将神经网络嵌入到图

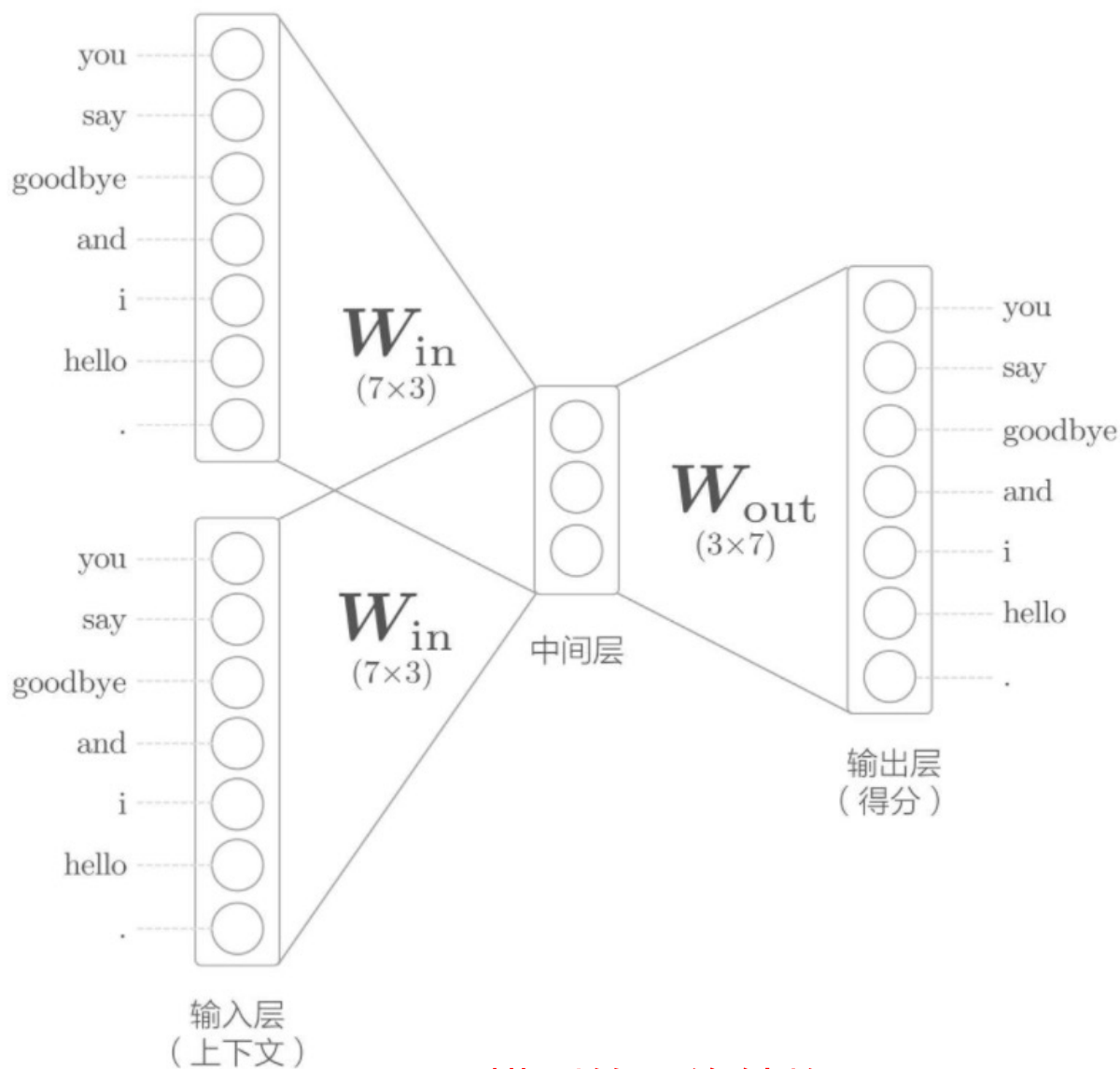


- ❑ 使用由原版word2vec提出的名为continuous bag-of-words (CBOW) 的模型作为神经网络
- ❑ word2vec一词最初用来指程序或者工具，但是随着该词的流行，在某些语境下，也指神经网络的模型
- ❑ CBOW模型和skip-gram模型是word2vec中使用的两个神经网络。

简单的word2vec--CBOW模型的推理

- ❑ CBOW模型:根据上下文预测目标词的神经网络
 - ❑ “目标词”：指中间的单词，“上下文”：周围的单词
- ❑ 通过训练这个CBOW模型，使其能尽可能地进行正确的预测，可以获得单词的分布式表示
- ❑ CBOW模型的输入是上下文。
 - ❑ 这个上下文用[‘you’, ‘goodbye’]这样的单词列表表示。将其转换为one-hot表示，以便CBOW模型可以进行处理。
- ❑ 在此基础上，CBOW模型的网络可以画成下图

简单的word2vec--CBOW模型的推理



CBOW模型的网络结构

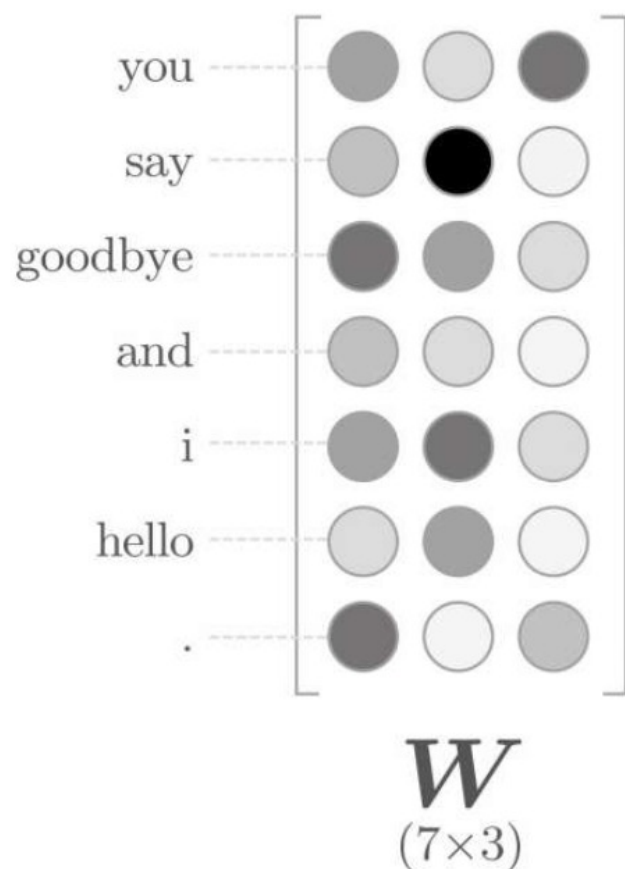
- 两个输入层，经过中间层到达输出层
- 权重 W_{in} ：输入层到中间层的变换由相同的全连接层
- 权重 W_{out} ：从中间层到输出层神经元的变换
- 上下文两个单词，输入层有两个。对应上下文

简单的word2vec--CBOW模型的推理

- **中间层**的神经元是各个输入层经全连接层变换后得到的值的“平均”。
- 就上面的例子而言，经全连接层变换后，第1个输入层转化为 h_1 ，第2个输入层转化为 h_2 ，那么中间层的神经元是 $\frac{1}{2}(h_1 + h_2)$
- **输出层**，这个输出层有7个神经元
 - 这里重要的是，这些神经元对应于各个单词。
 - 输出层的神经元是各个单词的得分，它的值越大，说明对应单词的出现概率就越高。
 - 得分是指在被解释为概率之前的值，对这些得分应用Softmax函数，就可以得到概率。
- 有时将得分经过Softmax层之后的神经元称为输出层。这里，我们将输出得分的节点称为输出层。

简单的word2vec--CBOW模型的推理

- ❑ 从输入层到中间层的变换由全连接层（权重是 W_{in} ）完成。
- ❑ 此时，全连接层的权重 W_{in} 是一个 7×3 的矩阵。这个权重就是我们要的单词的分布式表示

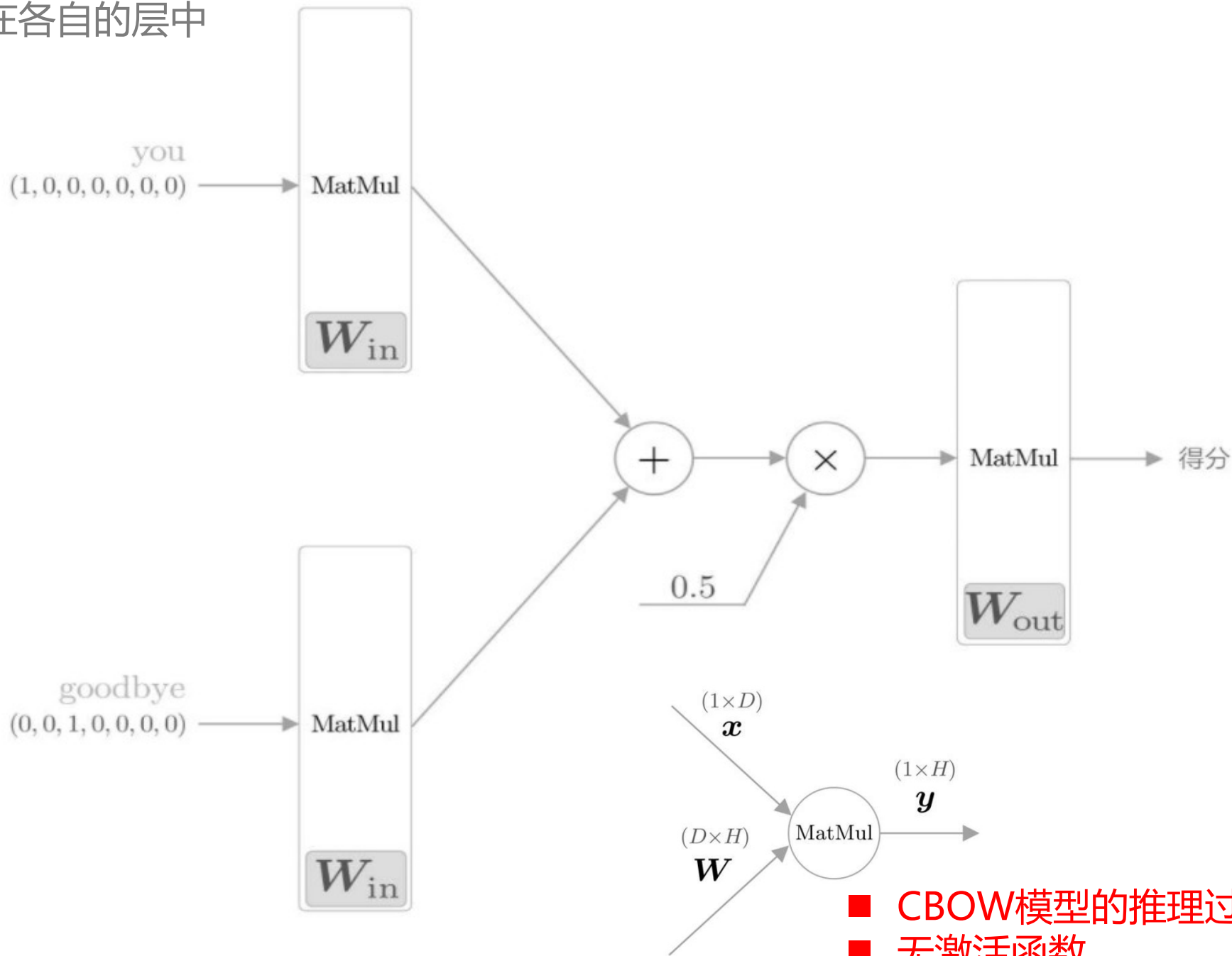


- 权重 W_{in} 的各行保存着各个单词的分布式表示。
- 通过反复学习，不断更新各个单词的分布式表示，以正确地从上下文预测出应当出现的单词。
- 令人惊讶的是，如此获得的向量很好地对单词含义进行了编码。
- 这就是word2vec的全貌。

简单的word2vec--CBOW模型的推理

- 中间层的神经元数量比输入层少这一点很重要。
- 中间层需要将预测单词所需的信息压缩保存，从而产生密集的向量表示。
- 中间层被写入了我们人类无法解读的代码，这相当于“编码”工作。
- 从中间层的信息获得期望结果的过程则称为“解码”。这一过程将被编码的信息复原为我们可以理解的形式。
- 从神经元视角图示了解CBOW模型。
- 下面，我们从层视角图示CBOW模型。这样一来，这个神经网络的结构如

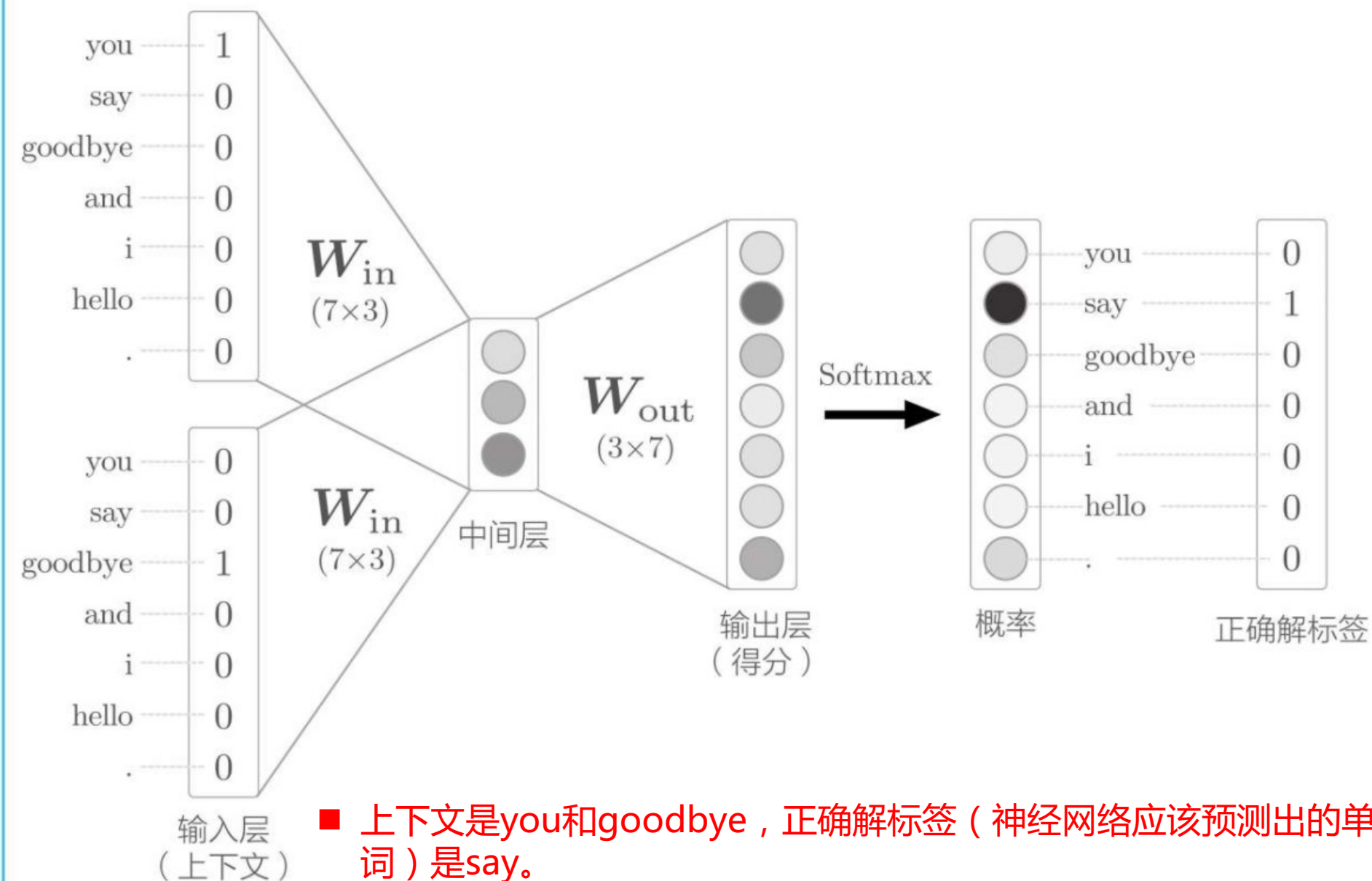
层视角下的CBOW模型的网络结构：MatMul层中使用的权重（ W_{in} 、 W_{out} ）画在各自的层中



- 基于推理的方法和神经网络
 - 基于计数的方法的问题
 - 基于推理的方法的概要
 - 神经网络中单词的处理方法
- 简单的word2vec
 - CBOW模型的推理
 - CBOW模型的学习
 - Word2vec的权重和分布式表示
- 学习数据的准备
 - 上下文和目标词
 - 转化为one-hot表示

简单的word2vec-CBOW模型的学习

- CBOW模型在输出层输出了各个单词的得分
- 通过对这些得分应用Softmax函数，可以获得概率
- 这个概率表示哪个单词会出现在给定的上下文(周围单词)中间



- 上下文是you和goodbye，正确解标签（神经网络应该预测出的单词）是say。
- 如果网络具有“良好的权重”，那么在表示概率的神经元中，对应正确解的神经元的得分应该更高

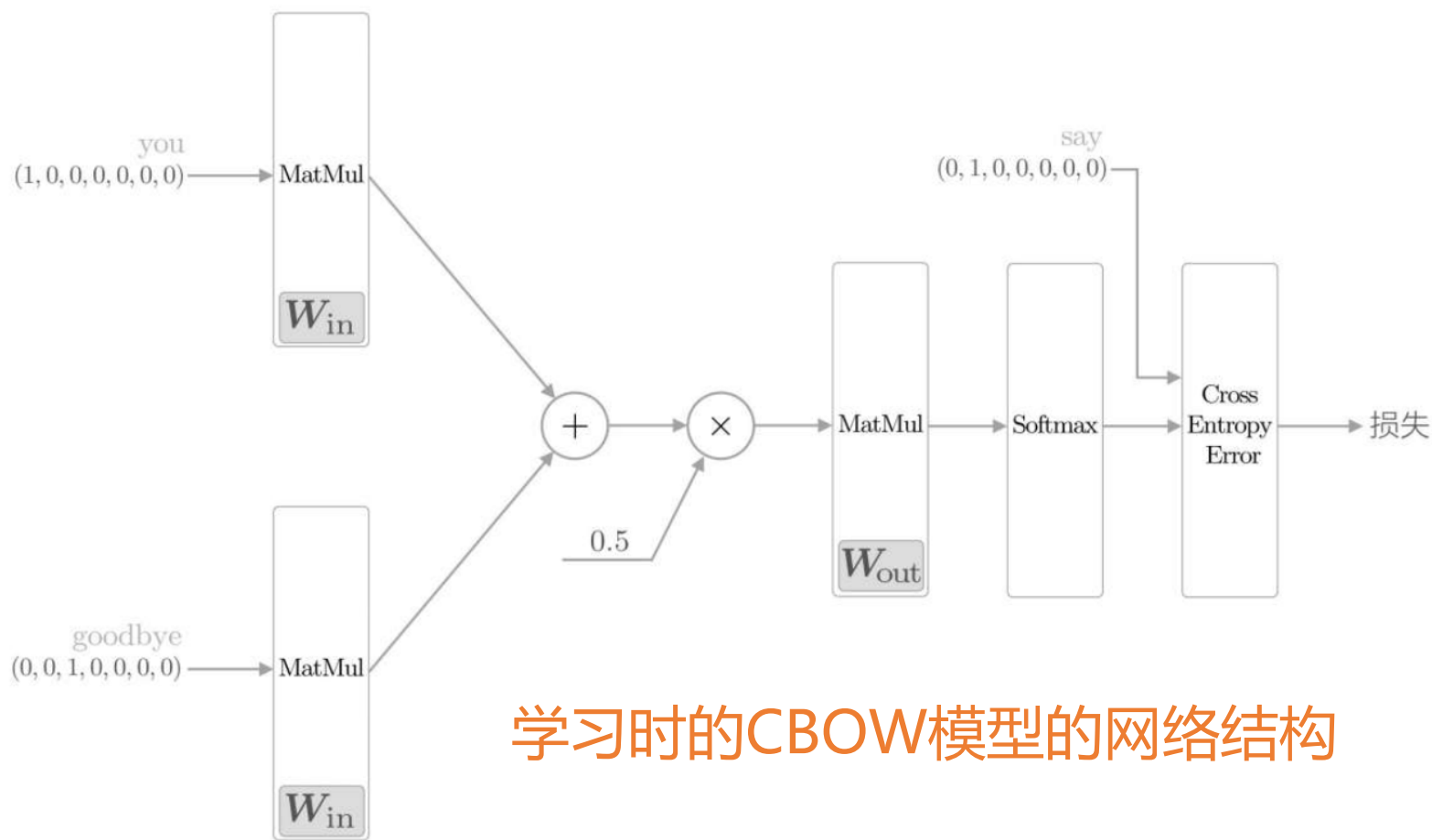
简单的word2vec-CBOW模型的学习

- ❑ CBOW模型的学习就是调整权重，以使预测准确。其结果是，权重 W_{in} (确切地说是 W_{in} 和 W_{out} 两者) 学习到蕴含单词出现模式的向量。
- ❑ 根据过去的实验，CBOW模型（和skip-gram模型）得到的单词的分布式表示，特别是使用维基百科等大规模语料库学习到的单词的分布式表示，在单词的含义和语法上符合我们直觉的案例有很多。
- ❑ CBOW模型只是学习语料库中单词的出现模式。如果语料库不一样，学习到的单词的分布式表示也不一样。

简单的word2vec-CBOW模型的学习

- 现在，我们来考虑一下上述神经网络的学习。
- 其实很简单，这里我们处理的模型是一个进行多类别分类的神经网络。
- 因此，对其进行学习只是使用一下Softmax函数和交叉熵误差。
- 首先，使用Softmax函数将得分转化为概率，再求这些概率和监督标签之间的交叉熵误差，并将其作为损失进行学习，这一过程可以用下图表示

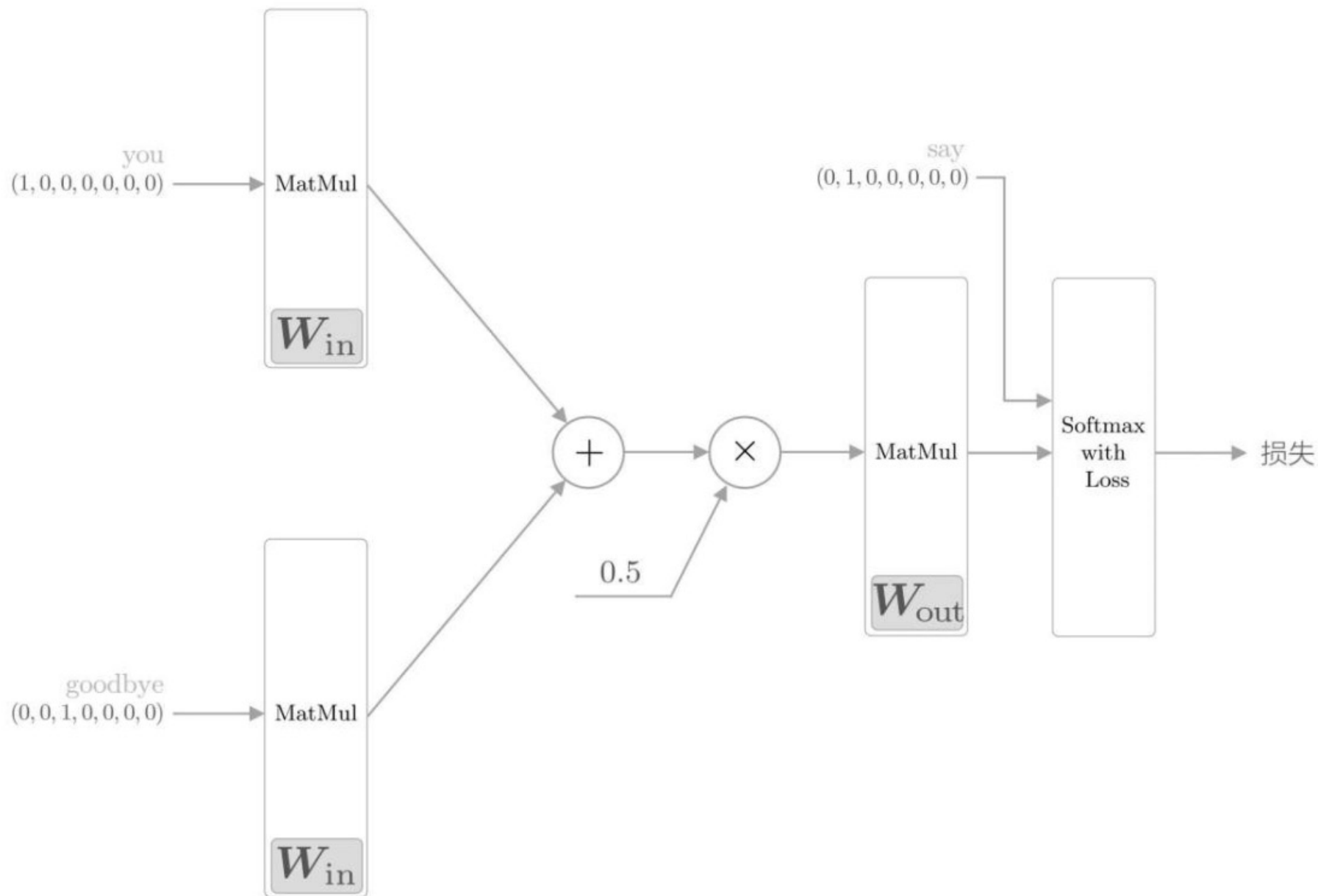
简单的word2vec--CBOW模型的学习



学习时的CBOW模型的网络结构

推理的CBOW模型加上Softmax层和Cross Entropy Error层，就可以得到损失。

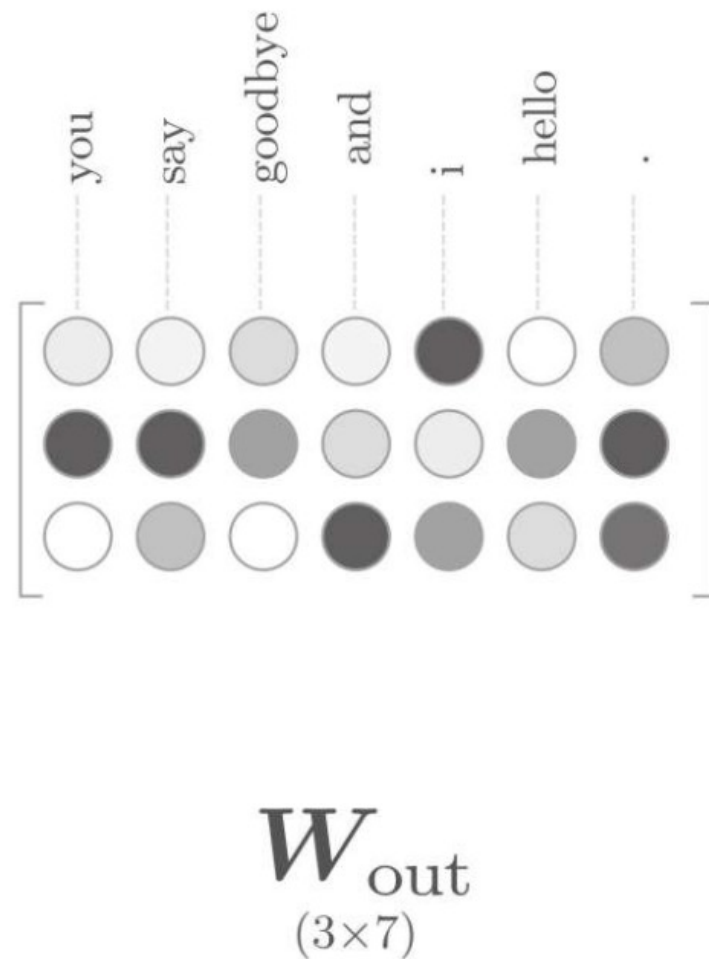
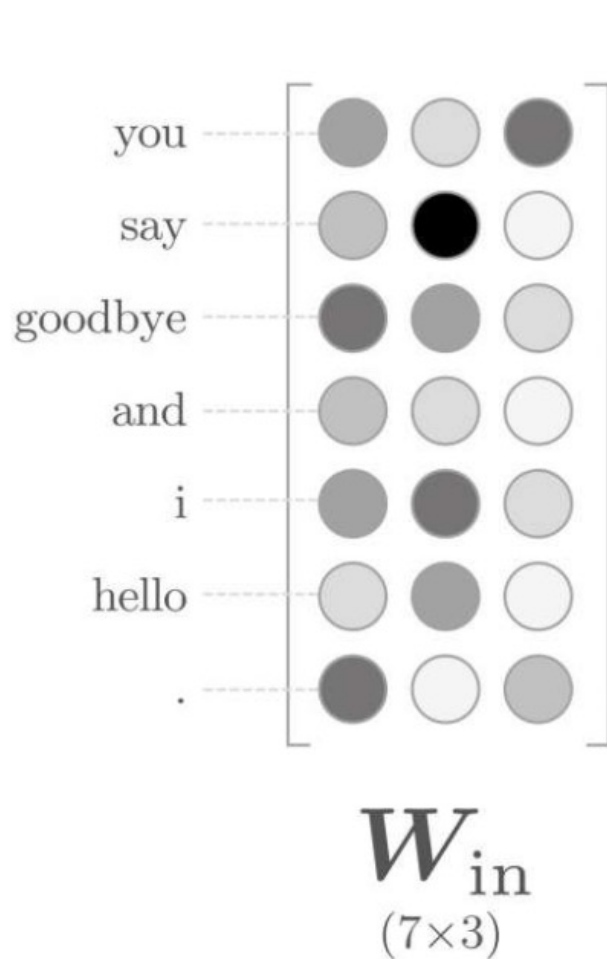
将Softmax层和Cross Entropy Error层统一为Softmax with Loss层



简单的word2vec-权重和分布式表示

- word2vec中使用的网络有两个权重，分别是：
 - ① 输入侧的全连接层的权重 W_{in}
 - ② 输出侧的全连接层的权重 W_{out}
- 输入侧的权重 W_{in} ：每一行对应于各个单词的分布式表示
- 输出侧的权重 W_{out} ：同样保存了对单词含义进行了编码的向量
- 如下图，输出侧的权重在列方向上保存了各个单词的分布式表示。

简单的word2vec-权重和分布式表示



输入侧和输出侧的权重都可以被视为单词的分布式表示

简单的word2vec-Word2vec的权重和分布式表示

那么，我们最终应该使用哪个权重作为单词的分布式表示呢？

- A. 只使用输入侧的权重
- B. 只使用输出侧的权重
- C. 同时使用两个权重

- 方案A和方案B只使用其中一个权重。
- 而在采用方案C的情况下，根据如何组合这两个权重，存在多种方式，其中一个方式就是简单地将这两个权重相加。

简单的word2vec-权重和分布式表示

- 就word2vec(特别是skip-gram模型)而言, 最受欢迎的是方案A
- 许多研究中也都会仅使用输入侧的权重 W_{in} 作为最终的单词的分布式表示。
- 文献通过实验证明了word2vec的skip-gram模型中 W_{in} 的有效性

学习数据的准备

- 在开始word2vec的学习之前，我们先来准备学习用的数据。
- 这里我们仍以 “You say goodbye and I say hello.” 这个只有一句话的语料库为例进行说明。

学习数据的准备-上下文和目标词

- word2vec中使用的神经网络的输入是上下文，它的正确解标签是被这些上下文包围在中间的单词，即目标词。
 - 即：当向神经网络输入上下文时，使目标词出现的概率高
- 从语料库生成上下文和目标词

corpus	contexts	target
you <u>say</u> goodbye and i say hello .	you, goodbye	say
you <u>say</u> <u>goodbye</u> <u>and</u> i say hello .	say, and	goodbye
you say <u>goodbye</u> <u>and</u> <u>i</u> say hello .	goodbye, i	and
you say goodbye <u>and</u> <u>i</u> <u>say</u> hello .	and, say	i
you say goodbye and i <u>say</u> <u>hello</u> .	i, hello	say
you say goodbye and i <u>say</u> <u>hello</u> .	say, .	hello

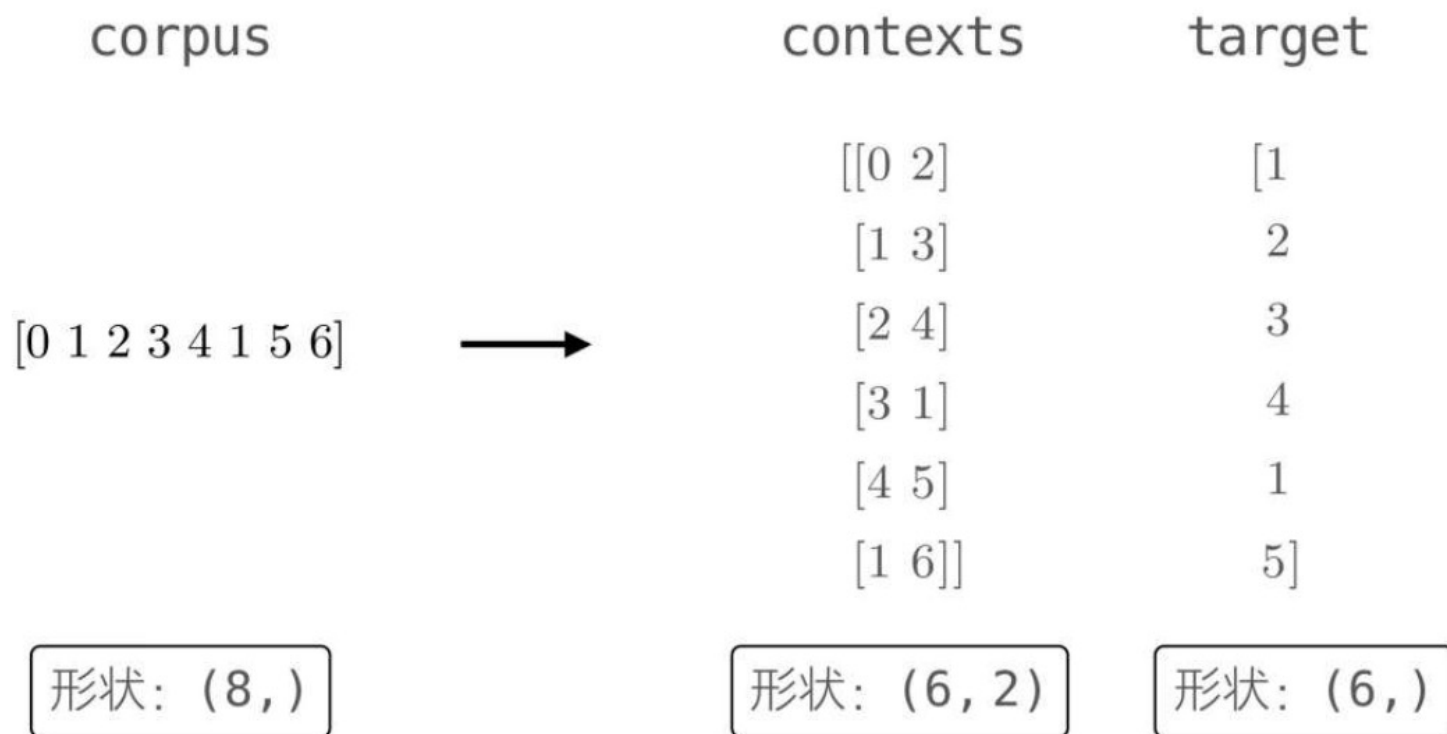
学习数据的准备-上下文和目标词

- ❑ 将语料库中的目标单词作为目标词，将其周围的单词作为上下文提取出来
- ❑ 对语料库中的所有单词都执行该操作(两端的单词除外) 得到 contexts(上下文) 和 target(目标词)
- ❑ contexts的各行成为神经网络的输入，target的各行成为正确解标签（要预测出的单词）

corpus	contexts	target
you <u>say</u> goodbye and i say hello .	you, goodbye	say
you <u>say</u> <u>goodbye</u> <u>and</u> i say hello .	say, and	goodbye
you say <u>goodbye</u> <u>and</u> i say hello .	goodbye, i	and
you say goodbye <u>and</u> <u>i</u> <u>say</u> hello .	and, say	i
you say goodbye and i <u>say</u> <u>hello</u> .	i, hello	say
you say goodbye and i <u>say</u> <u>hello</u> .	say, .	hello

学习数据的准备-上下文和目标词

- 将上下文和目标词转化成ID，再转化为one-hot表示
- 后面只需将它们赋给CBOW模型即可



学习数据的准备-上下文和目标词

corpus

you say goodbye and i say hello .
you say goodbye and i say hello .
you say goodbye and i say hello .
you say goodbye and i say hello .
you say goodbye and i say hello .
you say goodbye and i say hello .

contexts

[you, goodbye
say, and
goodbye, i
and, say
i, hello
say, .]

target

[say
goodbye
and
i
say
hello]

corpus

[0 1 2 3 4 1 5 6]

形状: (8,)

contexts

[[0 2]
[1 3]
[2 4]
[3 1]
[4 5]
[1 6]]

形状: (6, 2)

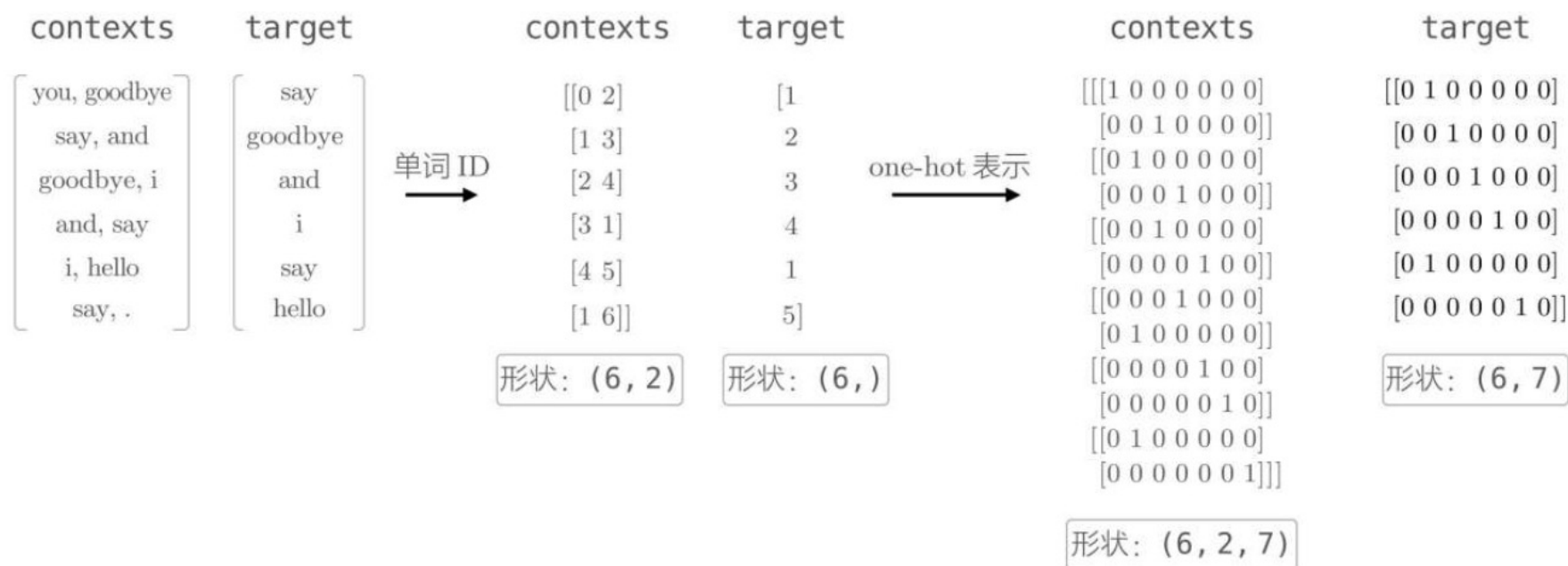
target

[1
2
3
4
1
5]

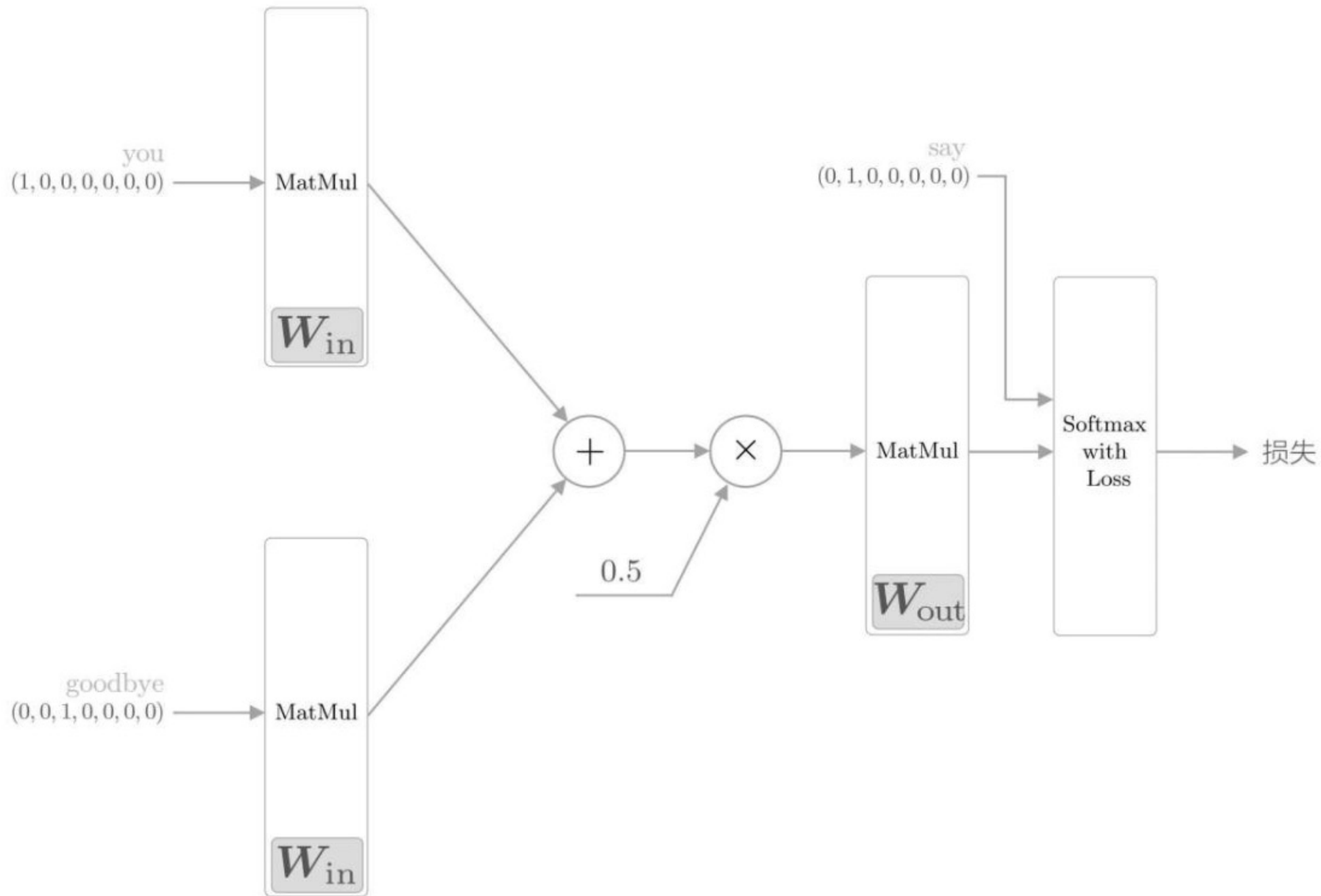
形状: (6,)

学习数据的准备-转化为one-hot表示

□ 上下文和目标词转化为one-hot表示的例子



- 使用单词ID时的contexts的形状是 (6,2) , 将其转化为one-hot表示后, 形状变为 (6,2,7) ——学习数据的准备就完成



- ❑ 基于推理的方法和神经网络
 - ❑ 基于计数的方法的问题
 - ❑ 基于推理的方法的概要
 - ❑ 神经网络中单词的处理方法
- ❑ 简单的word2vec
 - ❑ CBOW模型的推理
 - ❑ CBOW模型的学习
 - ❑ Word2vec的权重和分布式表示
- ❑ 学习数据的准备
 - ❑ 上下文和目标词
 - ❑ 转化为one-hot表示

- 简单的word2vec
 - CBOW模型的推理
 - CBOW模型的学习
 - word2vec的权重和分布式表示
- 学习数据的准备
 - 上下文和目标词
 - 转化为one-hot表示
- word2vec的补充说明
 - CBOW模型和概率
 - skip-gram模型
 - 基于计数与基于推理

word2vec的补充说明

- 详细探讨了word2vec的CBOW模型
- 接下来，我们将对word2vec补充说明几个非常重要的话题。
- 首先，我们从概率的角度，再来看一下CBOW模型。

word2vec的补充说明——CBOW模型和概率

标记说明：

- ▣ 将概率记为 $P(\cdot)$ ，比如事件A发生的概率记为 $P(A)$ 。
- ▣ 联合概率记为 $P(A, B)$ ，表示事件A和事件B同时发生的概率。
- ▣ 后验概率记为 $P(A|B)$ ，字面意思是“事件发生后的概率”
 - ▣ 从另一个角度来看，也可以解释为“在给定事件B(信息) 时事件A发生的概率”。

word2vec的补充说明——CBOW模型和概率

概率的表示方法来描述CBOW模型

- CBOW模型进行的处理是：当给定某个上下文时，输出目标词的概率。
- 使用包含单词 $w_1, w_2, \dots, w_T$ 的语料库。如下图，对第 t 个单词，考虑窗口大小为1的上下文。

$$w_1 \ w_2 \ \cdots \ \underbrace{w_{t-1} \ \boxed{w_t} \ w_{t+1}} \ \cdots \ w_{T-1} \ w_T$$

word2vec的CBOW模型：从上下文的单词预测目标词

word2vec的补充说明——CBOW模型和概率

- 用数学式来表示当给定上下文 w_{t-1} 和 w_{t+1} 时目标词为 w_t 的概率。使用后验概率，有：

$$P(w_t | w_{t-1}, w_{t+1})$$

- 上式表示 “在 w_{t-1} 和 w_{t+1} 发生后， w_t 发生的概率”
- 也可以解释为 “当给定 w_{t-1} 和 w_{t+1} 时， w_t 发生的概率”
- 也就是说，CBOW模型可以建模为上式

word2vec的补充说明——CBOW模型和概率

CBOW模型的损失函数：

□ 交叉熵误差函数 $L = - \sum_k t_k \log y_k$

□ y_k 表示第k个事件发生的概率。 t_k 是监督标签，它是one-hot向量的元素。这里需要注意的是，“ w_t 发生”这一事件是正确解，它对应的one-hot向量的元素是1，其他元素都是0。考虑到这一点，可以推导出下式：

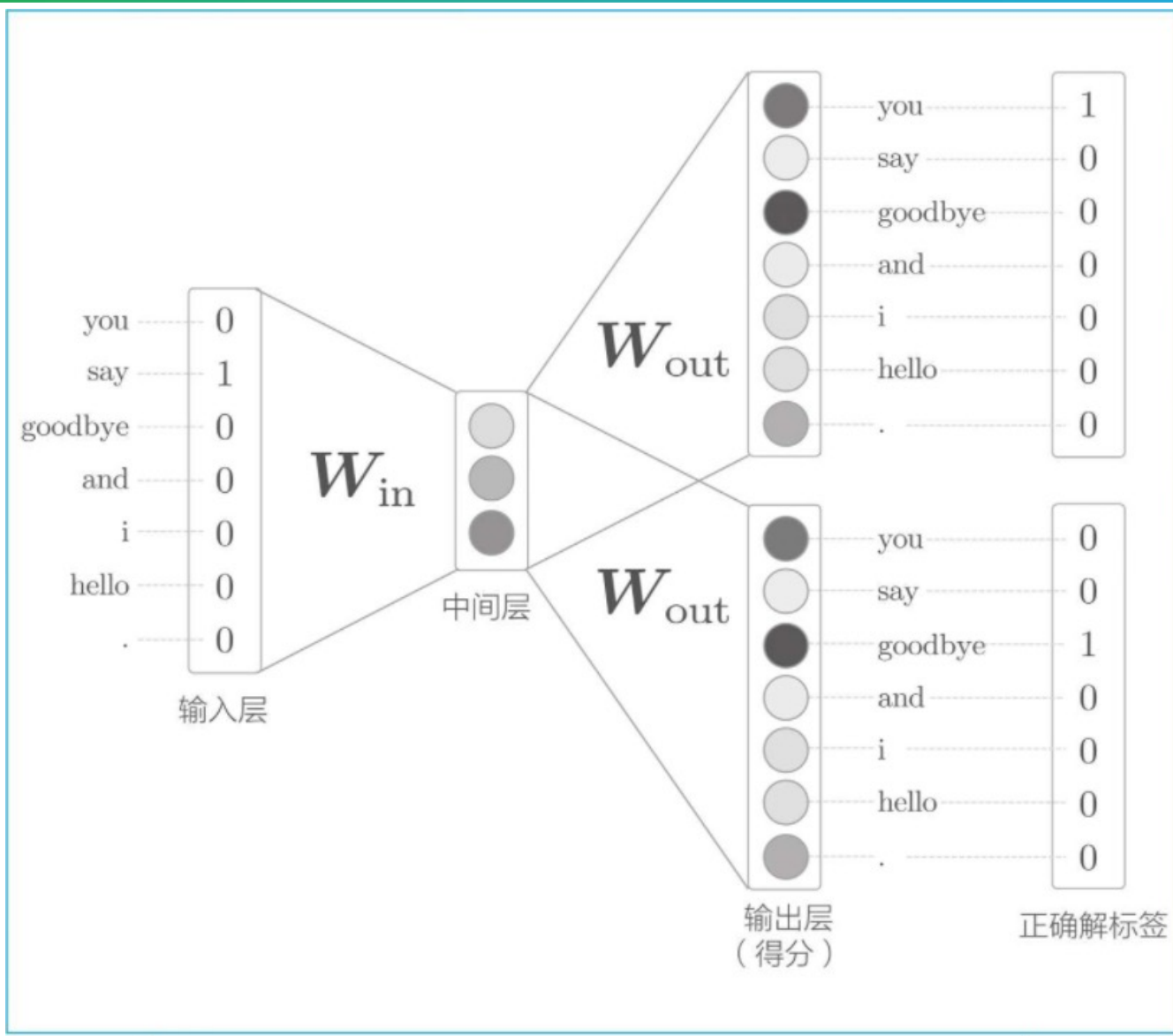
$$L = -\log P(w_t | w_{t-1}, w_{t+1})$$

□ 其扩展到整个语料库，则损失函数可以写为

$$L = -\frac{1}{T} \sum_{t=1}^T \log P(w_t | w_{t-1}, w_{t+1})$$

CBOW模型学习的任务让上式表示的损失函数尽可能地小。那时的权重参数就是我们想要的单词的分布式表示。

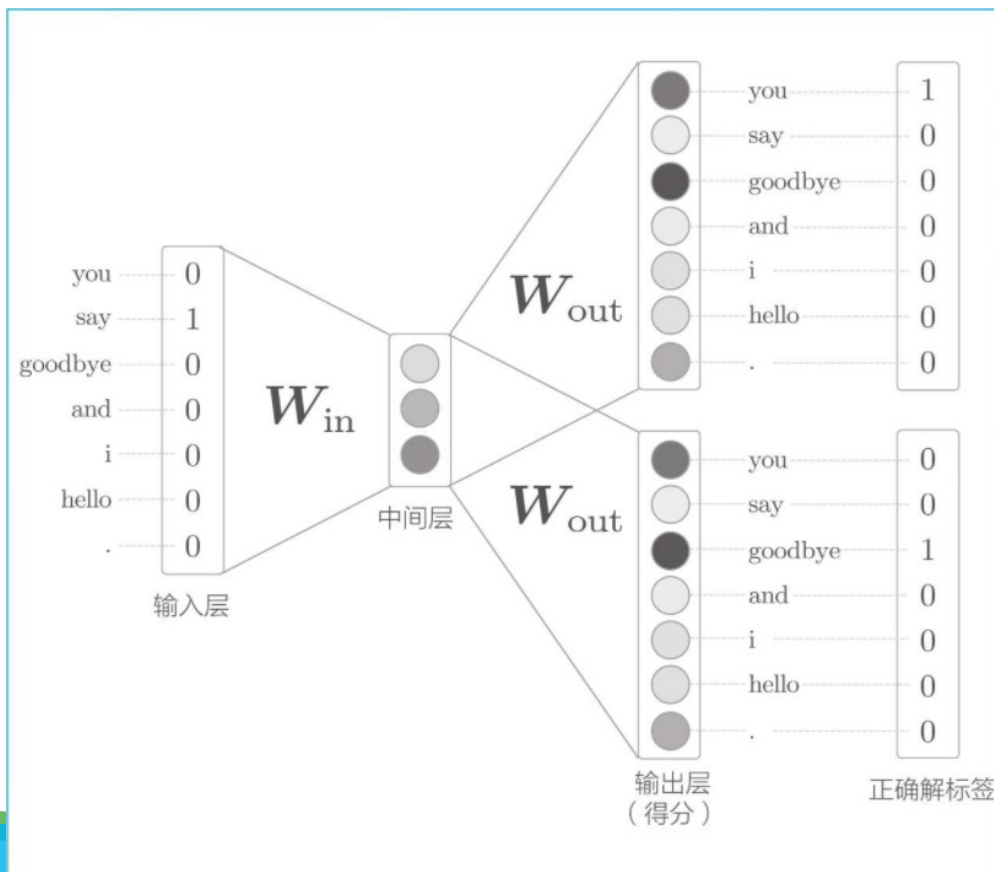
word2vec的补充说明——skip-gram模型



skip-gram
模型的例子

word2vec的补充说明——skip-gram模型

- ❑ skip-gram模型的输入层只有一个，输出层的数量则与上下文的单词个数相等
- ❑ 要分别求出各个输出层的损失（通过Softmax with Loss层等），然后将它们加起来作为最后的损失



word2vec的补充说明——skip-gram模型

使用概率的表示方法来表示skip-gram模型

□考虑根据中间单词（目标词） w_t 预测上下文 w_{t-1} 和 w_{t+1} 的情况。此时，skip-gram可以建模为

$$P(w_{t-1}, w_{t+1} | w_t)$$

□表示“当给定 w_t 时， w_{t-1} 和 w_{t+1} 同时发生的概率”

□在skip-gram模型中，假定上下文的单词之间**没有相关性**（正确地说是假定“条件独立”），上式进行分解：

$$P(w_{t-1}, w_{t+1} | w_t) = P(w_{t-1} | w_t) P(w_{t+1} | w_t)$$

word2vec的补充说明——skip-gram模型

代入交叉熵误差函数，可以推导出skip-gram模型的损失函数：

$$\begin{aligned} L &= -\log P(w_{t-1}, w_{t+1} | w_t) \\ &= -\log P(w_{t-1} | w_t) P(w_{t+1} | w_t) \\ &= -(\log P(w_{t-1} | w_t) + \log P(w_{t+1} | w_t)) \end{aligned}$$

扩展到整个语料库，则skip-gram模型的损失函数可以表示为式

$$L = -\frac{1}{T} \sum_{t=1}^T (\log P(w_{t-1} | w_t) + \log P(w_{t+1} | w_t))$$

skip-gram模型的损失函数需要求各个上下文单词对应的损失的总和。而CBOW模型只需要求目标词的损失

word2vec的补充说明——skip-gram模型

应该使用CBOW模型和skip-gram模型中的哪一个呢？

skip-gram模型

- ▣ **单词的分布式表示的准确度**: 在大多数情况下，skip-gram模型的结果更好
 - ▣ 特别是随着语料库规模的增大，在低频词和类推问题的性能方面，skip-gram模型往往会有更好的表现
- ▣ **学习速度**: CBOW模型比skip-gram模型要快
 - ▣ 原因：skip-gram模型需要根据上下文数量计算相应个数的损失，计算成本变大
- ▣ skip-gram模型根据一个单词预测其周围的单词，这是一个非常难的问题。经过这个更难的问题的锻炼，skip-gram模型能提供更好的单词的分布式表示。

word2vec的补充说明——基于计数与基于推理

- 基于计数的方法和基于推理的方法（特别是word2vec）。
- 两种方法在学习机制上存在显著差异：
 - 基于计数的方法通过对整个语料库的统计数据进行一次学习来获得单词的分布式表示
 - 基于推理的方法则反复观察语料库的一部分数据进行学习（mini-batch学习）
- 我们就其他方面来对比一下这两种方法

word2vec的补充说明——基于计数与基于推理

① 考虑需要向词汇表添加新词并更新单词的分布式表示的场景

- 基于计数的方法：需要从头开始计算

- 即便是想稍微修改一下单词的分布式表示，也需要重新完成生成共现矩阵、进行SVD等一系列操作。

- 基于推理的方法(word2vec)：允许参数的增量学习

- 可以将之前学习到的权重作为下一次学习的初始值，在不损失之前学习到的经验的情况下，高效地更新单词的分布式表示

- 在这方面，基于推理的方法（word2vec）具有优势

② 两种方法得到的单词的分布式表示的性质和准确度有何差异？

- 基于计数的方法：主要是编码单词的相似性

- Word2vec(特别是skip-gram模型)：除了单词的相似性以外，还能理解更复杂的单词之间的模式

- word2vec因能解开“king-man+woman=queen”这样的类推问题而知名

word2vec的补充说明—基于计数与基于推理

- 基于推理的方法在**准确度**方面优于基于计数的方法？
 - 有研究表明，就**单词相似性的定量**评价而言：基于推理的方法和基于计数的方法难分上下
 - 基于推理的方法和基于计数的方法**存在关联性**：使用了skip-gram和Negative Sampling的模型被证明与对整个语料库的共现矩阵进行特殊矩阵分解的方法具有相同的作用。故，这两个方法论(在某些条件下)是“相通”的
 - GloVe方法**融合**了基于推理的方法和基于计数的方法：
 - 在word2vec之后，有研究人员提出了GloVe方法
 - 该方法的**思想**是：将整个语料库的统计数据的信息纳入损失函数，进行mini-batch学习。据此，这两个方法论成功地被融合在了一起。

- 简单的word2vec
 - CBOW模型的推理
 - CBOW模型的学习
 - word2vec的权重和分布式表示
- 学习数据的准备
 - 上下文和目标词
 - 转化为one-hot表示
- word2vec的补充说明
 - CBOW模型和概率
 - skip-gram模型
 - 基于计数与基于推理

Word2vec高速化

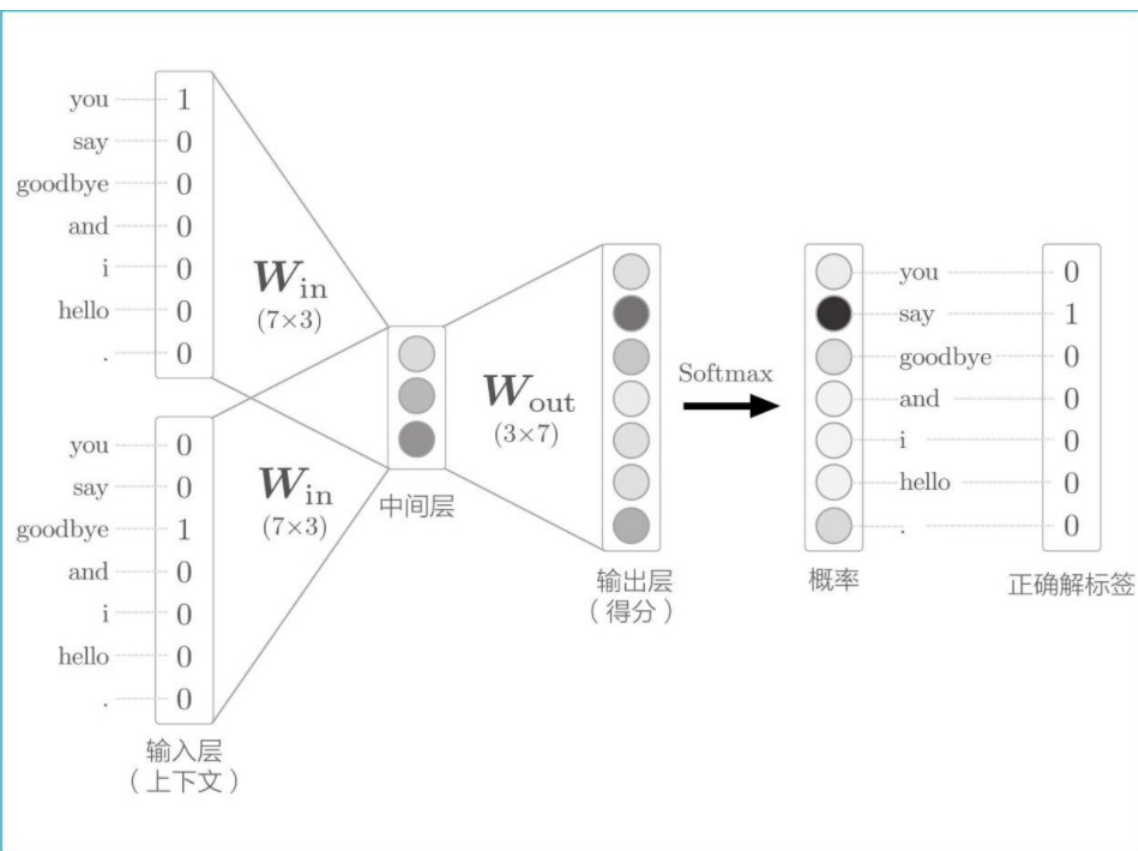
- ❑ 改进一 ——Embedding层
- ❑ 改进二
 - ❑ 中间层之后的计算问题
 - ❑ 从多分类到二分类
 - ❑ Sigmoid函数和交叉熵误差
 - ❑ 多分类到二分类的实现
 - ❑ 负采样
 - ❑ 负采样的采样方法
 - ❑ 负采样的实现
- ❑ 改进版word2vec的学习
- ❑ word2vec的其他话题

Word2vec高速化

- ❑ CBOW模型：一个简单的2层神经网络，实现起来比较简单
- ❑ CBOW模型最大的问题：随着语料库中处理的词汇量的增加，计算量也随之增加。实际上，当词汇量达到一定程度之后，CBOW模型的计算就会花费过多的时间。
- ❑ 重点放在word2vec的加速上，来改善word2vec。具体而言，进行两点改进：
 - ① 引入名为Embedding层的新层
 - ② 以及引入名为Negative Sampling的新损失函数

CBOW模型问题

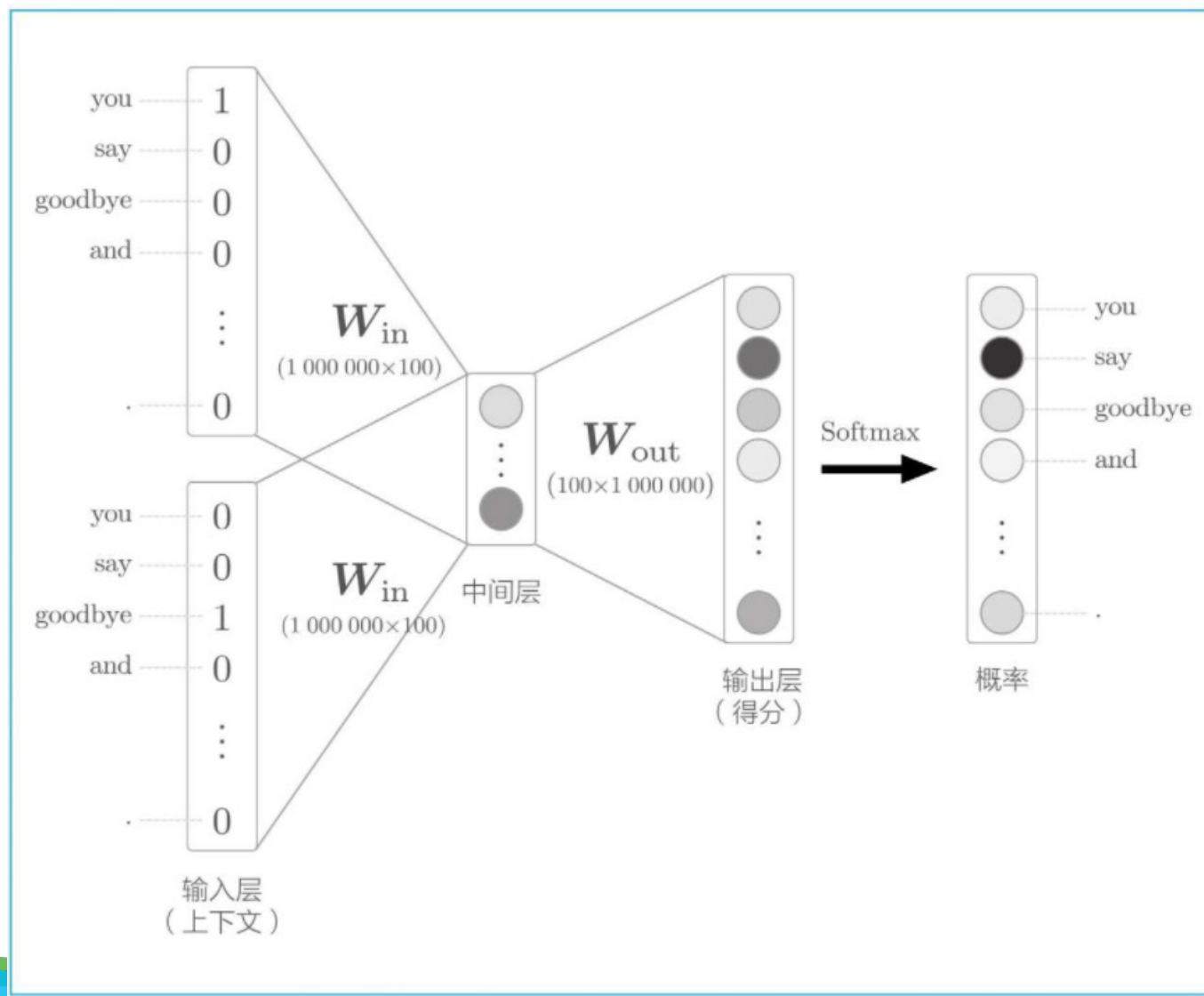
■ CBOW模型接收拥有2个单词的上下文，并基于它们预测1个单词



- ① 通过输入层和输入侧权重(W_{in})之间的矩阵乘积计算中间层
- ② 通过中间层和输出侧权重(W_{out})之间的矩阵乘积计算每个单词的得分
- ③ 得分经过Softmax函数转化后，得到每个单词的出现概率
- ④ 概率与正确解标签进行比较计算出损失

CBOW模型问题

- ❑ CBOW模型在处理小型语料库时问题不大



假设词汇量为100万个时的CBOW模型

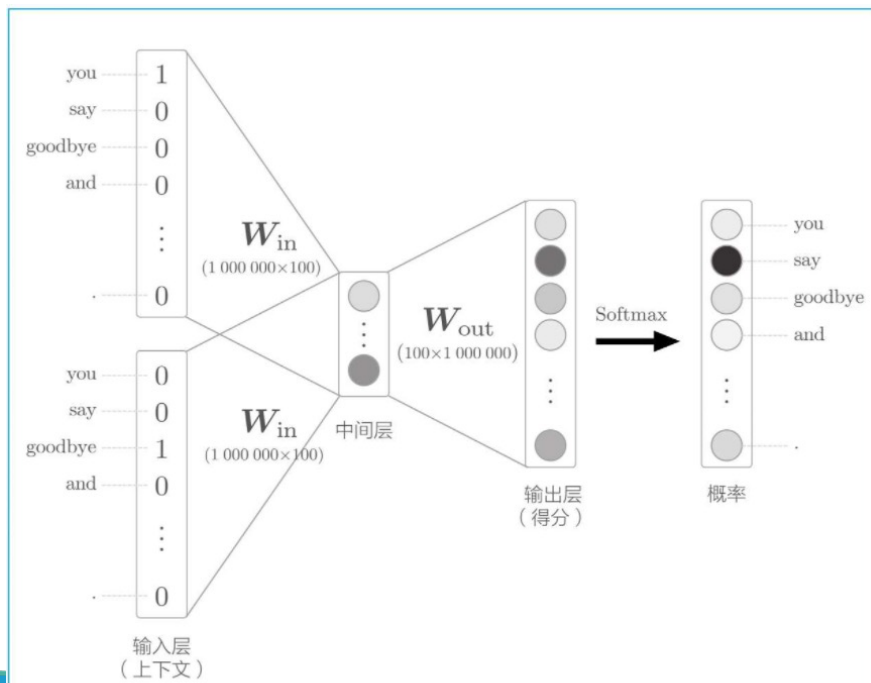
CBOW模型问题

I. 输入层的one-hot表示：

- ① 仅one-hot表示本身就需要占用100万个元素的内存大小
- ② 还需要计算one-hot表示和权重矩阵 W_{in} 的乘积

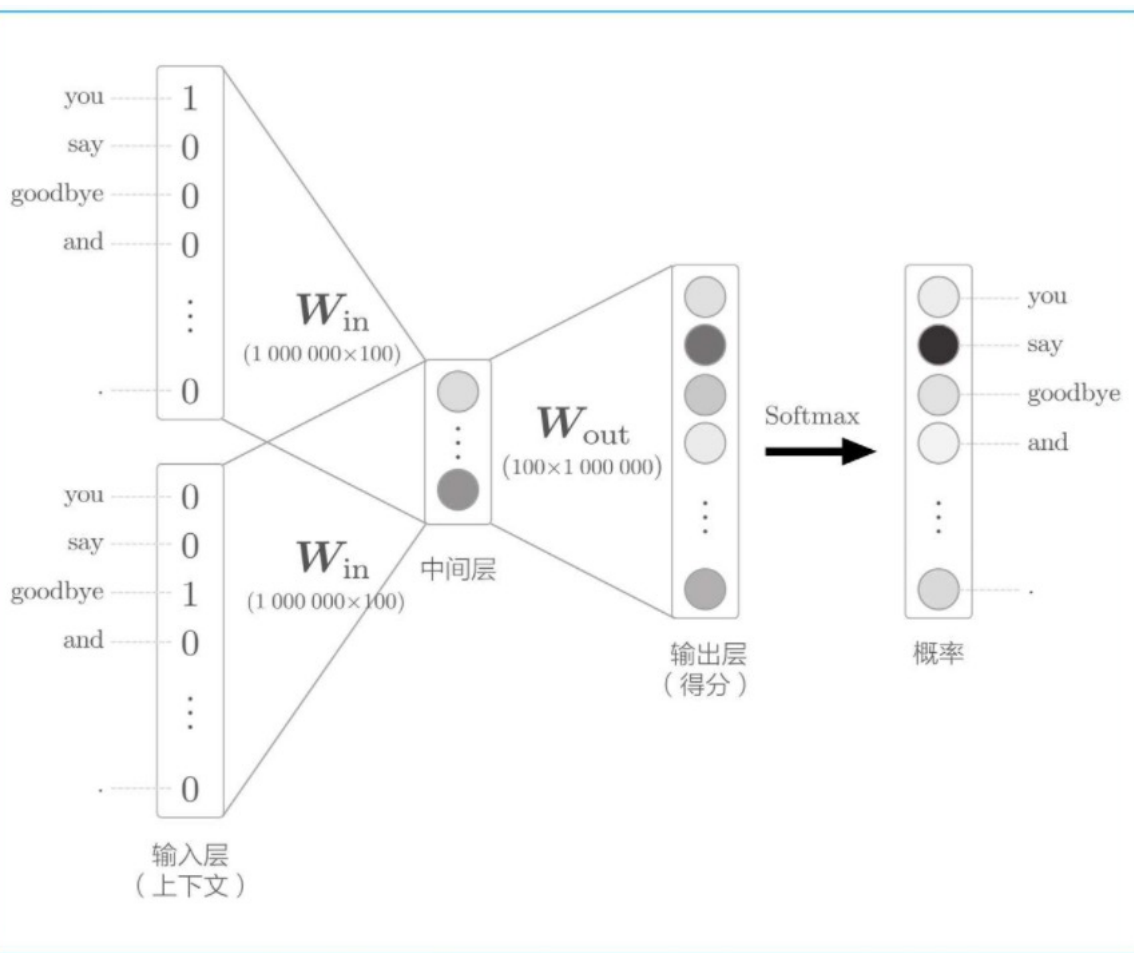
II. 中间层和权重矩阵 W_{out} 的乘积以及Softmax层的计算

- ① 中间层和权重矩阵 W_{out} 的乘积需要大量的计算
- ② Softmax层的计算量



word2vec的改进①——Embedding层

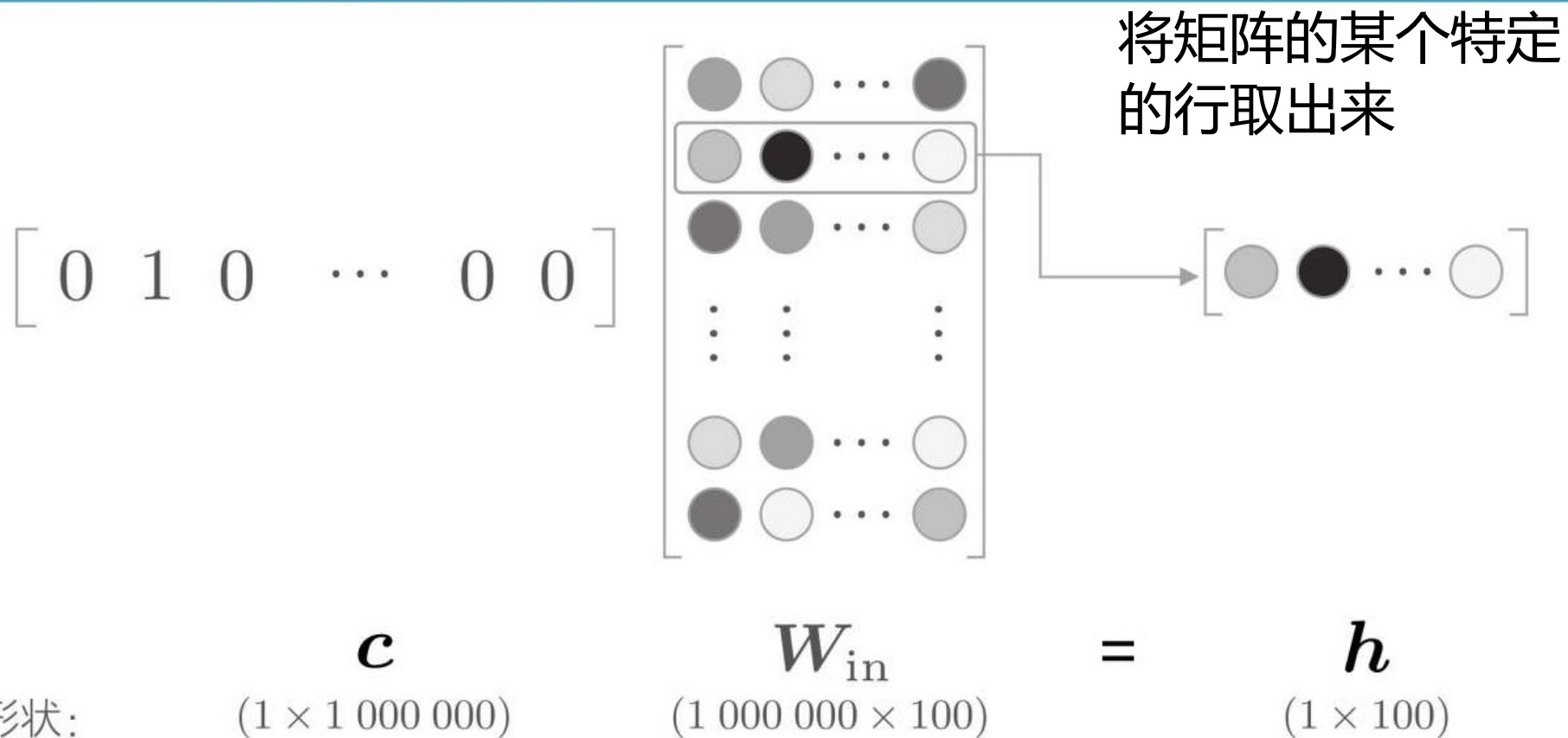
- 输入层和输出层存在100万个神经元，计算会出现瓶颈



- ① 输入层的one-hot表示和权重矩阵 W_{in} 的乘积
- ② 中间层和权重矩阵 W_{out} 的乘积以及Softmax层的计算

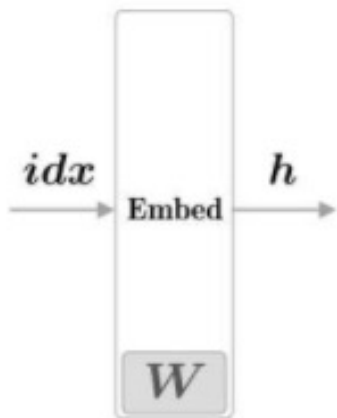
word2vec的改进①——Embedding层

one-hot表示的上下文和 W_{in} 层的权重的乘积

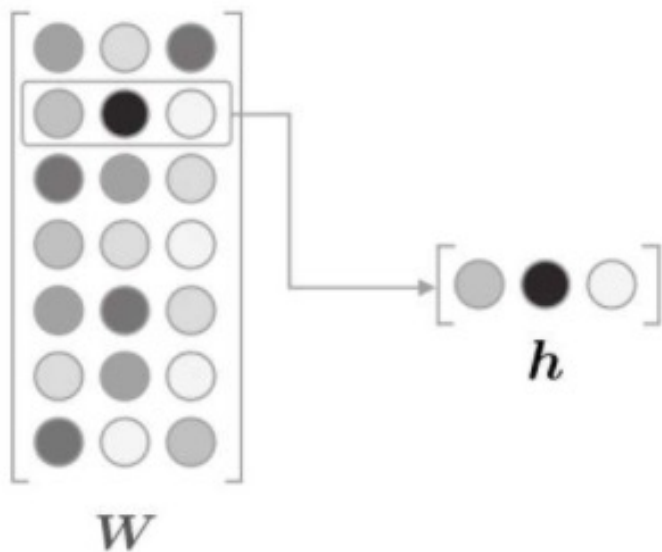


创建一个从权重参数中抽取“单词ID对应行”的层——Embedding层

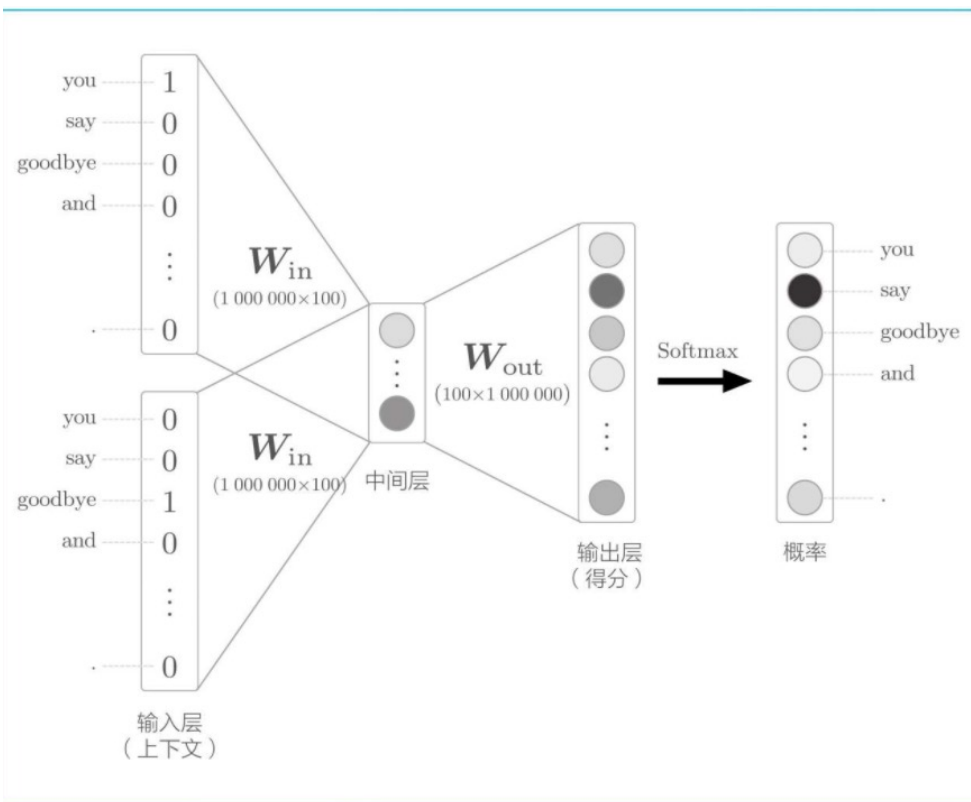
word2vec的改进②



- Embedding层记为Embed
- 单词的密集向量表示称为词嵌入(word embedding)或者单词的分布式表示(distributed representation)



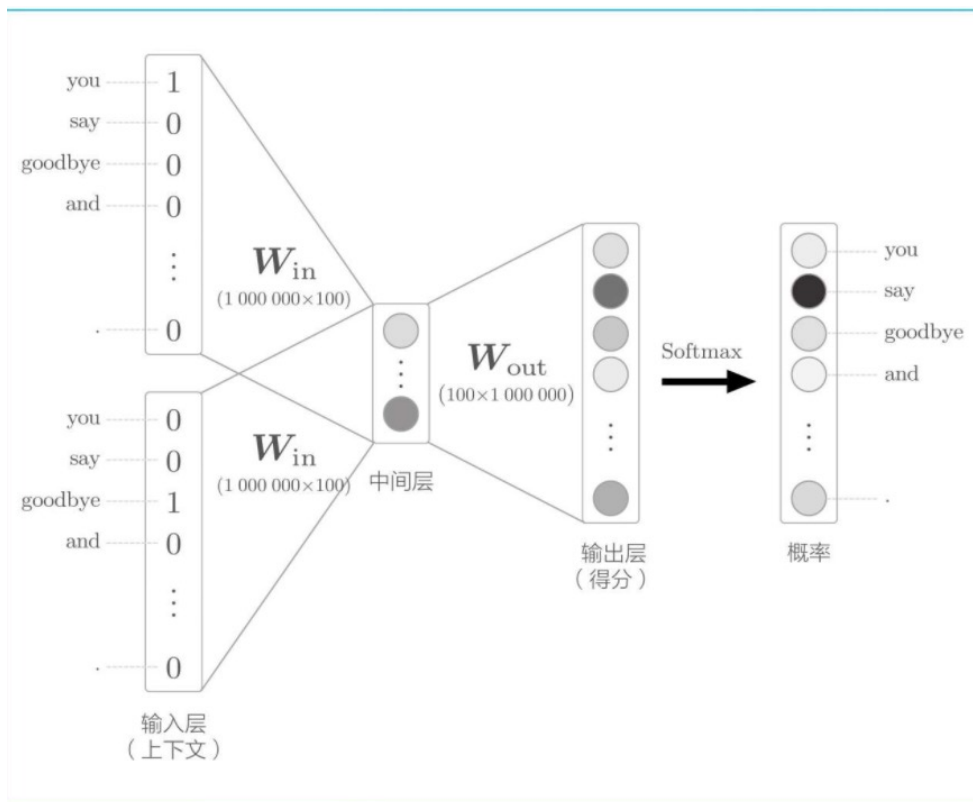
word2vec的改进②



- ❑ 矩阵乘积和Softmax层的计算
- ❑ 权重矩阵的大小是 100×1000000 万，计算需要大量时间、内存
- ❑ 100万次

$$y_k = \frac{\exp(s_k)}{\sum_{i=1}^{1\,000\,000} \exp(s_i)}$$

word2vec的改进②



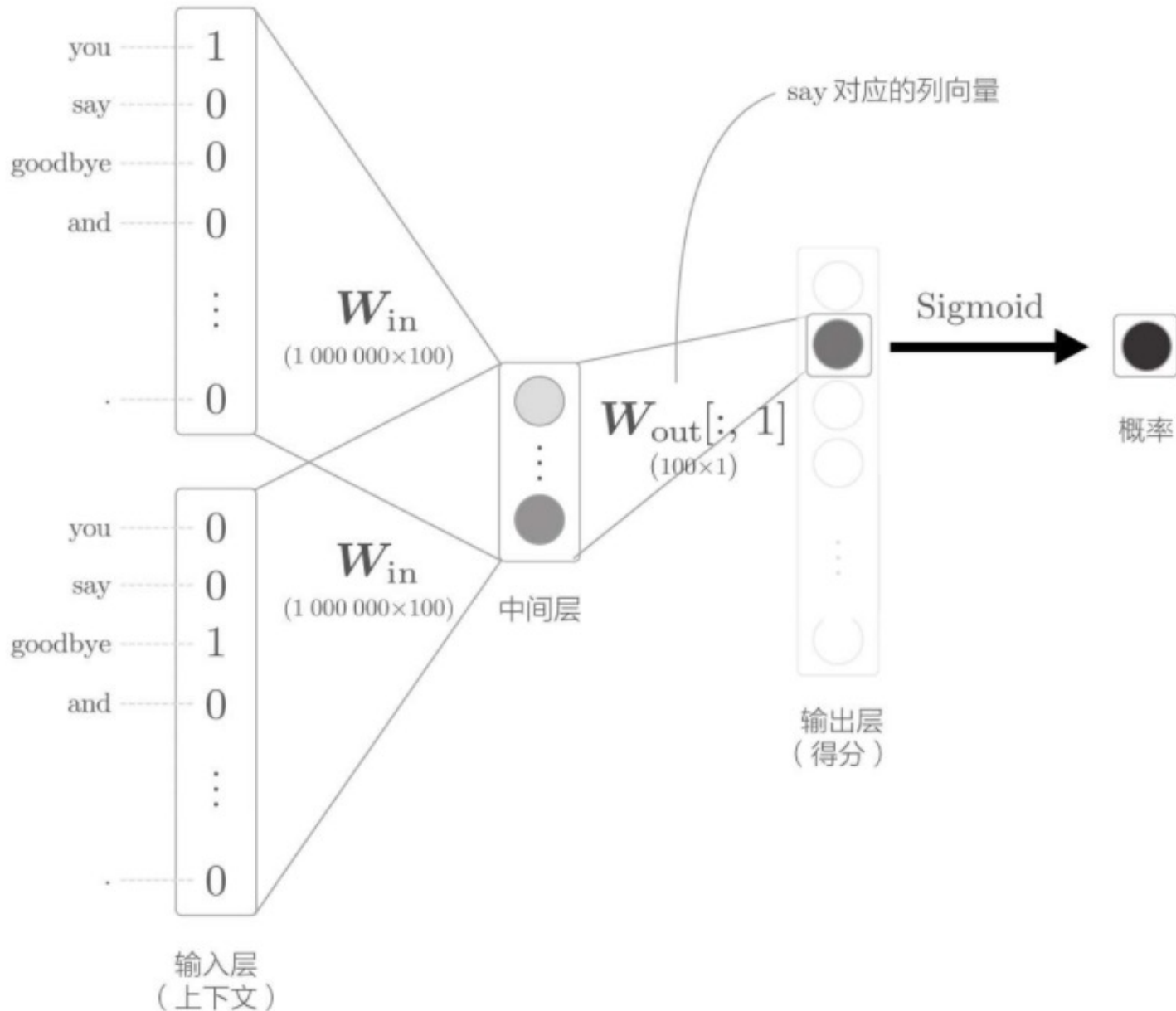
- 矩阵乘积和Softmax层的计算
- 负采样(negative sampling)的方法
- 使用Negative Sampling替代Softmax，无论词汇量有多大，都可以使计算量保持较低或恒定。

word2vec的改进② ——从多分类到二分类

关键思想在于：二分类拟合多分类

- ▣ 多分类：当上下文是you和goodbye时，目标词是什么？
- ▣ 二分类：当上下文是you和goodbye时，目标词是say吗？
 - ▣ 输出层只需要一个神经元，输出的是say的得分
- ▣ CBOW模型进行什么样的处理呢？

仅计算目标词的得分的神经网络

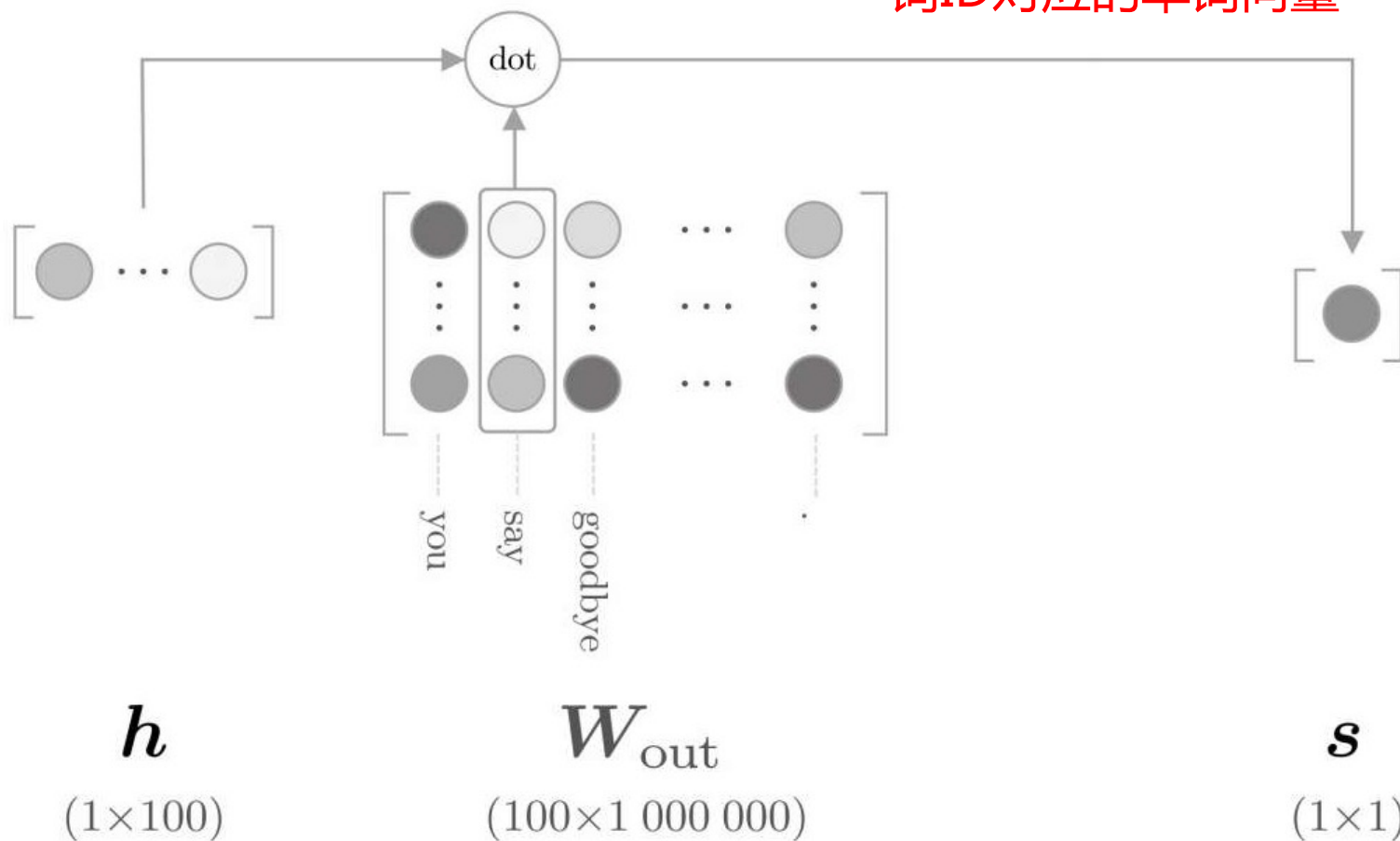


word2vec的改进② —— 多分类到二分类的实现

只需要提取say对应的列(单词向量), 并用它与中间层的神经元计算内积即可

sigmoid函数将其转化为概率

W_{out} 中保存了各个单词ID对应的单词向量



word2vec的改进② —— Sigmoid函数和交叉熵误差

- 输出层使用sigmoid函数，损失函数也使用交叉熵误差

- sigmoid函数

$$y = \frac{1}{1 + \exp(-x)}$$

- sigmoid函数呈S形，输入值x被转化为0到1之间的实数,输出y可以解释为概率

- 交叉熵误差 $L = -(t \log y + (1-t) \log (1-y))$

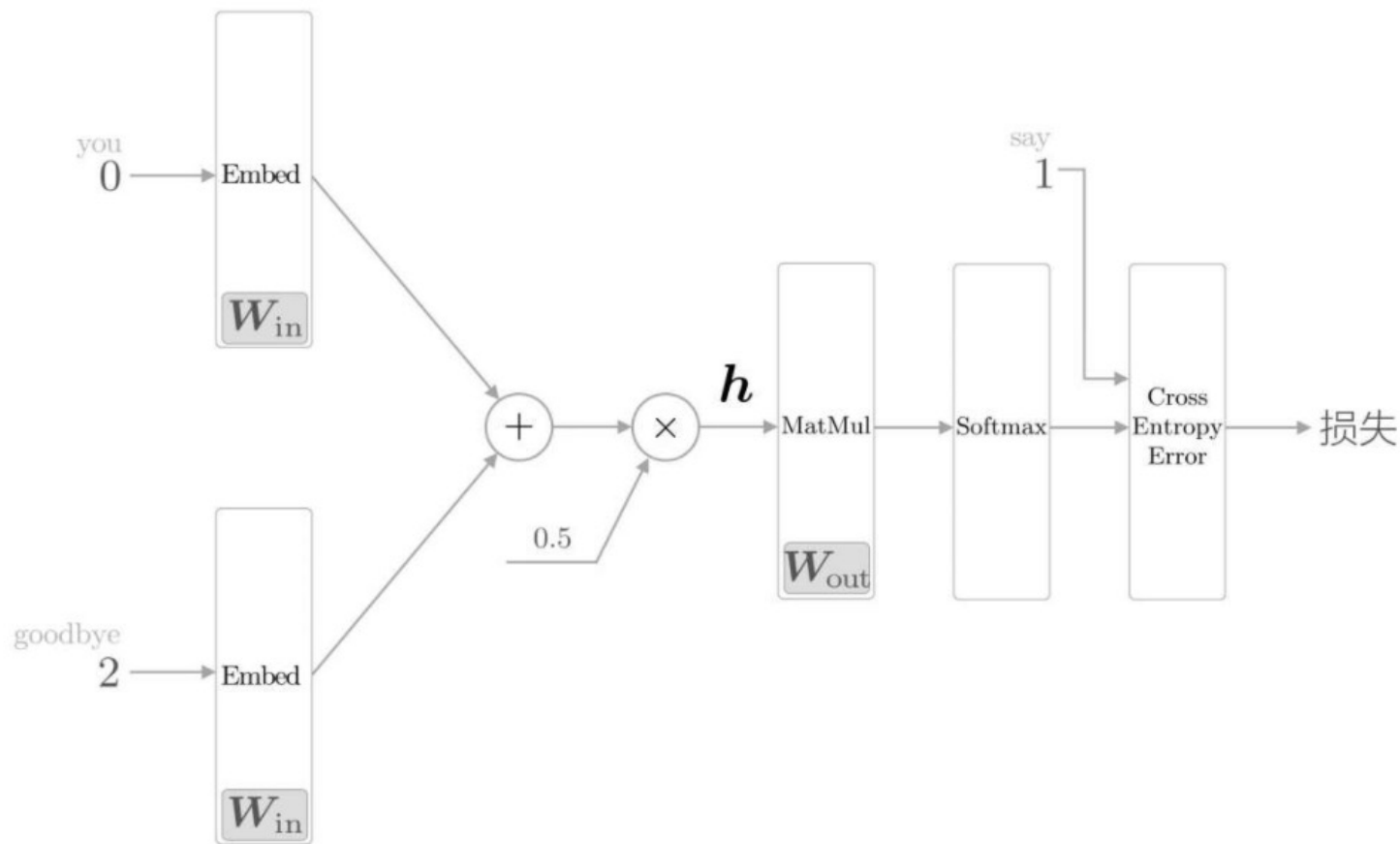
- y : sigmoid函数的输出

- t : 正确解标签，取值为0或1

- 取值为1时: 表示正确解是 “Yes”
- 取值为0时: 表示正确解是 “No”
- 当t为1时,输出: $-\log y$
- 当t为0时,输出: $-\log (1-y)$

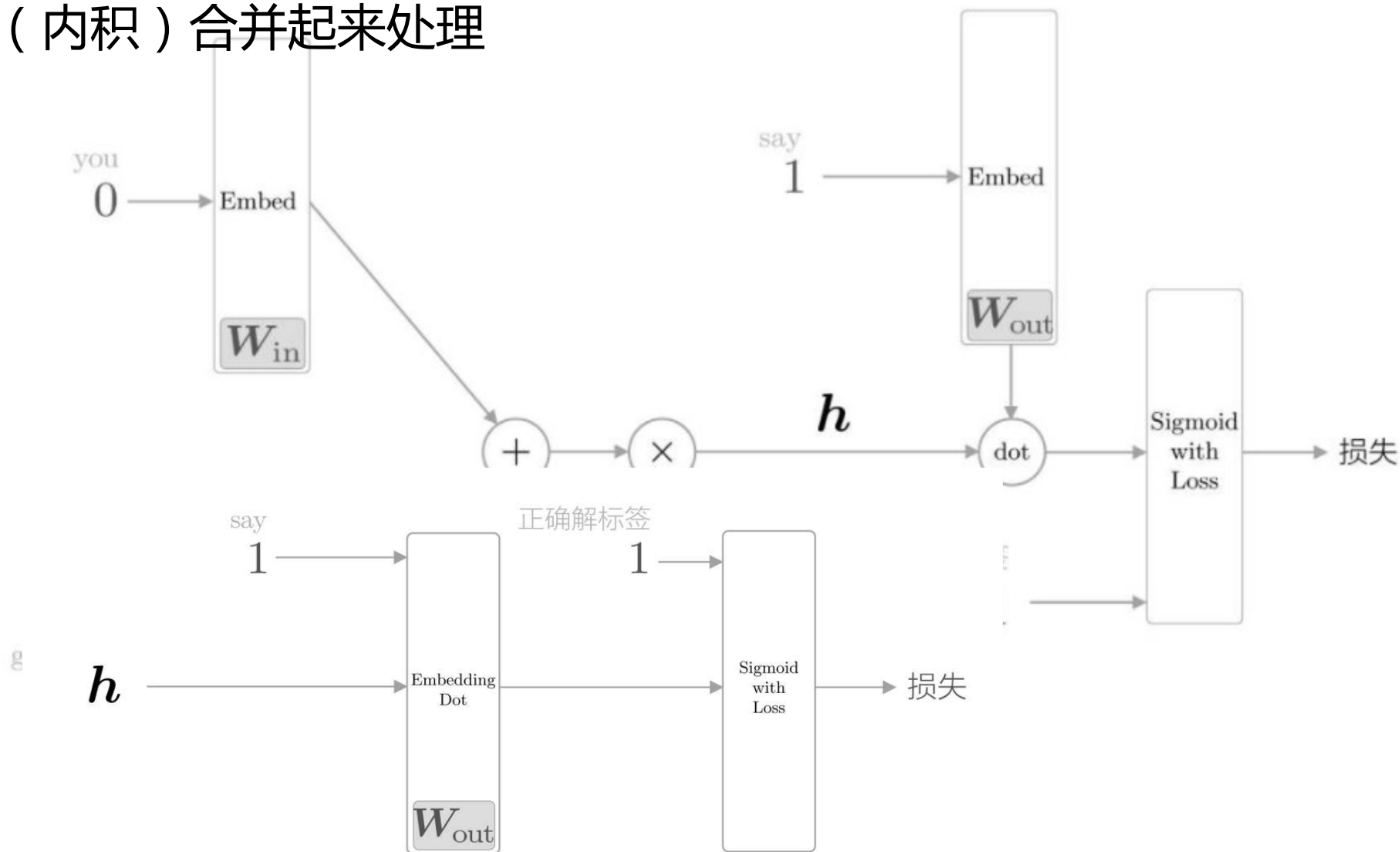
word2vec的改进② ——Sigmoid函数和交叉熵误差

进行多分类的CBOW模型的全貌图（Embedding层记为Embed）

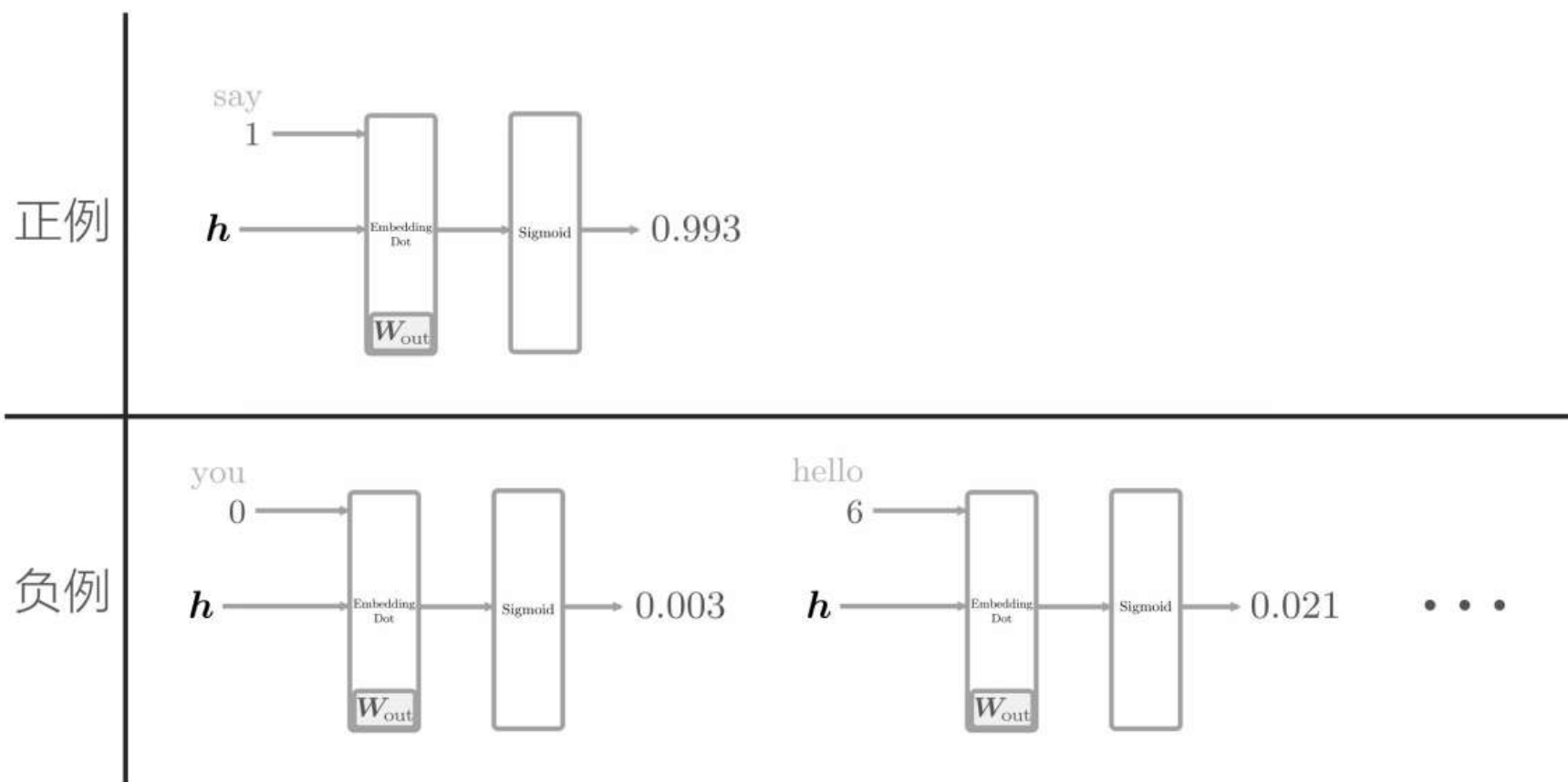


进行二分类的CBOW模型的全貌图

引入Embedding Dot层，该层将Embedding层和dot运算（内积）合并起来处理



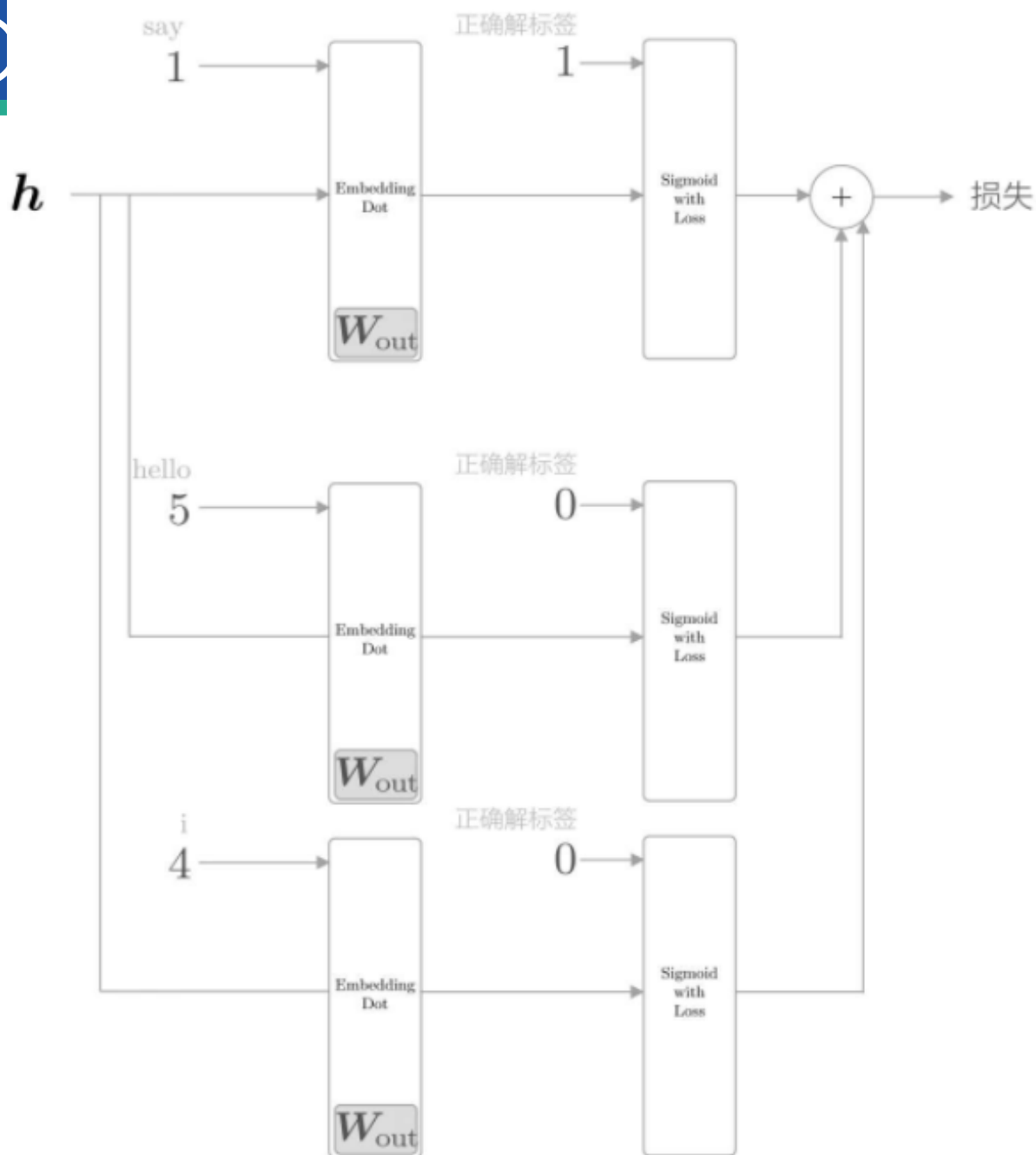
word2vec的改进② ——负采样



- ❑ 需要以所有的负例为对象进行学习吗？NO 词汇量将暴增
- ❑ 选择若干个（5个或者10个）负采样方法

word2vec的改进②

- 对正例和负例的处理
- 正例 (say) 和之前一样，向Sigmoid with Loss层输入正确解标签1；
- 负例 (hello和i) 是错误答案，所以要向Sigmoid with Loss层输入正确解标签0。
- 将各个数据的损失相加，作为最终损失输出。



word2vec的改进② ——负采样的采样方法

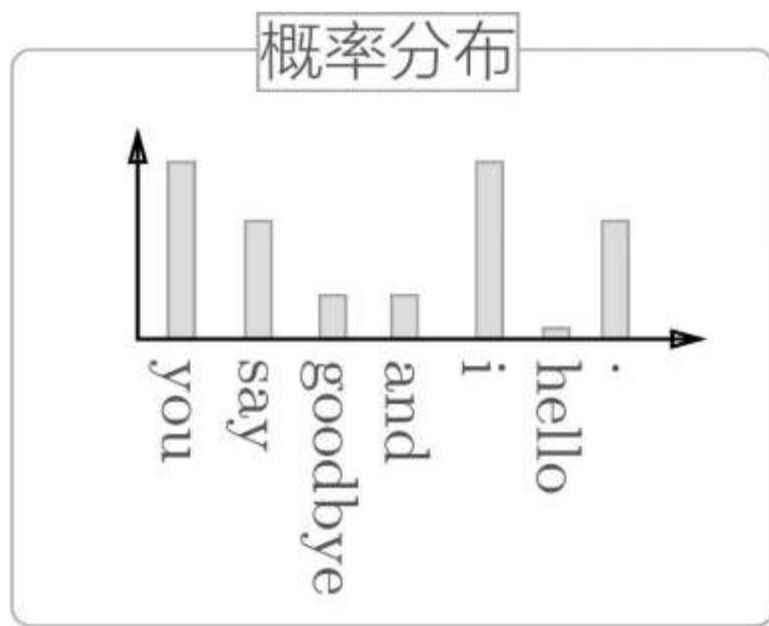
□ 如何抽取负例？

□ 随机抽样？

□ 基于语料库的统计数据进行采样？

□ 经常出现的单词容易被抽到，不经常出现的单词难以被抽到

□ 各个单词的出现次数求出概率分布后，根据概率分布进行采样



采样

→ “you”
→ “i”
→ “.”
→ “goodbye”
→ “you”

Thanks!
Q&A ?

wuxianyan@njau.edu.cn